CS336 · Spring 2026 · Lecture 03

语言模型架构与超参数：从残差通路到推理成本

如何阅读这份自学增强讲义

原始英文幻灯片完整保留；每组原页之后紧接对应中文讲解。书签可跳转到原页与讲解。页码中的“原页”始终指原始 PDF 页码。

中文部分为自学补充；课堂转述、原论文结论与编者推导应按文中标记区分。第三方保存的字幕仅用于检索定位，不等同于教师审校逐字稿。

学习目标：读完本讲，应能从张量形状写出一个现代 decoder-only Transformer，解释 Pre-Norm、RMSNorm、SwiGLU 与 RoPE 的动机，估算 FFN 参数和 KV cache 大小，并区分训练、prefill 与逐 token 解码的成本。遇到训练尖峰时，应能分辨数值溢出、logit 尺度失控及优化问题；面对模型配置表，应知道哪些是维度约束、哪些只是经验默认值。

先修：矩阵乘法与转置、链式法则、均值/方差、softmax 与交叉熵，以及第2讲的 FLOPs、内存带宽和算术强度。阅读主线是“保留易优化的残差通路 → 为每个 token 提供非线性 → 向注意力注入相对位置 → 在参数预算下选形状 → 控制训练尺度 → 为生成减少数据搬运”。正文用 d 表示模型宽度，m 表示 FFN 中间宽度，h 表示查询头数，k 表示每头维度，g 表示 K/V 头数，L 表示层数；序列长度与批量分别为 n、b。

版本与证据：本材料对应 Stanford CS336 Spring 2026 第3讲，原67页按顺序完整保留。中文是相邻教学注释，JSON 仅作可重建源。公式推演和数字算例均为编者补充；“课堂补充”仅转述已核对的第三方录像字幕，并注明时间，不将字幕视为教师审校稿。模型名单和趋势是课件制作时的样本，不代表持续更新的全行业统计。原稿疑似排印或算术问题会明确标注，不改原页。

Lecture 3

Everything you didn't want to know about LM architecture and hyperparameters

CS336

Tatsu H

Outline and goals

- Quick recap of a modern transformer (what you implement)
- What do most of the large LMs have in common?
- What are common variations to the architecture / training process?

Today's theme: the best way to learn is hands-on experience
the second best way is to try to learn from others' experience

第 3 讲 · 原页 1–2 · 中文讲解

如何使用这一讲：架构选择是带约束的实验

本讲要回答的问题

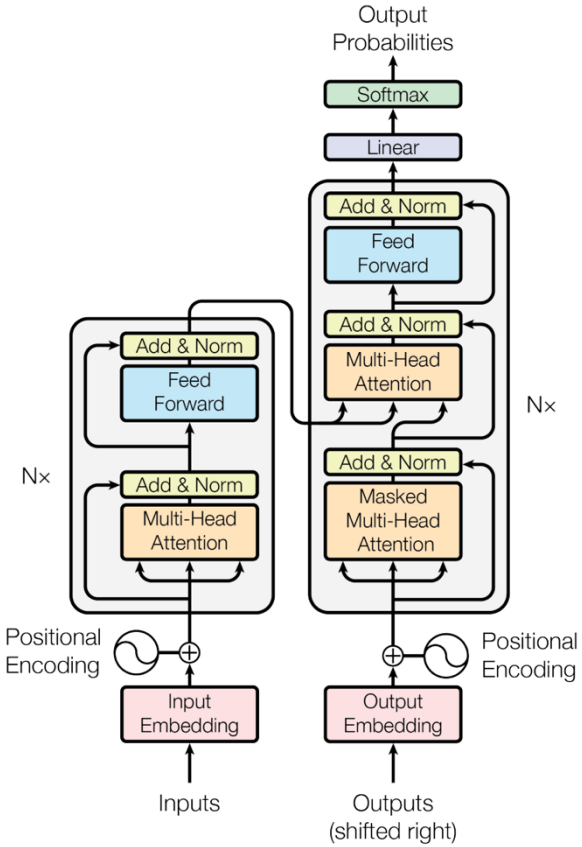
第1讲说明语言模型要学习什么，第2讲说明张量计算要花多少资源，这一讲把二者连接起来：为什么同样是预测下一个 token，模型会选不同的归一化、激活函数、位置编码与头数？“可以运行”“训练稳定”“质量更好”“运行更快”是四种不同结论。某结构被很多模型采用，只能说明它是有经验支持的起点，不能替代同预算的消融实验。

编者补充：带着三个账本阅读

第一个账本记形状：每一层输入输出必须能连接。第二个记资源：参数、激活、KV cache、FLOPs 与字节搬运。第三个记证据：理论是在何种假设下成立，实验是在何种数据、规模与硬件上得到。比如 SwiGLU 的质量比较要对齐参数量，而 GQA 的好处主要发生在自回归解码；把两者只用“FLOPs 少”来解释，会错过本讲的核心。读图时先找横纵轴、比较对象与控制变量，再读作者的结论。

本节参考（完整书目见讲义末尾）：[slides]

Starting point: the ‘original’ transformer



Review: choices in the standard transformer

Position embedding: sines and cosines

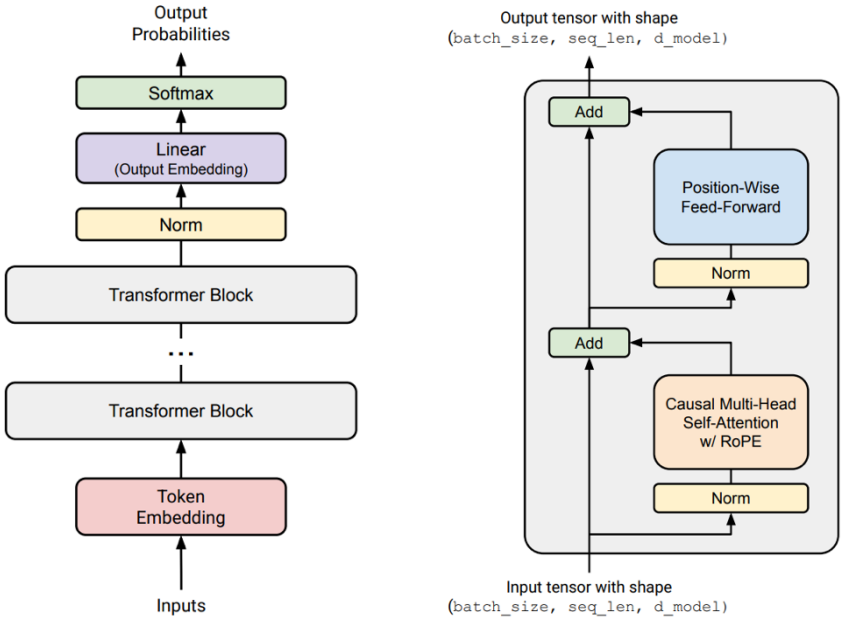
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

FFN: ReLU

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Norm type: post-norm, LayerNorm

What you implemented – simple, modern variant



Differences:

- **LayerNorm** is in front of the block
- **Rotary position embeddings (RoPE)**
- FF layers use **SwiGLU**, not ReLU
- Linear layers (and layernorm) have **no bias** (constant) terms

Why did we pick these?

What should you pick?

第 3 讲 · 原页 3–4 · 中文讲解

从原始 **Transformer** 到现代语言模型

先把整条计算链串起来

第3页的原始 Transformer 是 encoder-decoder：编码器读输入，解码器既做因果自注意力，也对编码器输出做交叉注意力。第4页的课程模型是 decoder-only：token ID → 嵌入 → 多层因果自注意力与 FFN → 最终归一化 → 词表 logits。每个位置用前缀预测下一个 token；训练可用 causal mask 同时处理整段，生成时才有逐步依赖。不要把原图中的 cross-attention 搬进课程 decoder-only 模型。

编者补充：一个块的形状与公式

设 X 的形状为 [b,n,d]。一个 Pre-Norm 串行块可写成 U = X + Attention(Norm(X))，Y = U + FFN(Norm(U))；每个加号两侧都是 [b,n,d]。注意力先用 W_Q、W_K、W_V 将最后一维映射到头空间，按头计算 softmax($QK^T/\sqrt{k}$ + M)V，再用输出投影回到 d；M 对未来位置加负无穷。若 q、k 各坐标独立、零均值、方差约为1，点积方差约为 k，除以 √k 可让其尺度不随头维度直接增长。这是初始化附近的尺度动机，训练后并不自动满足独立同分布。

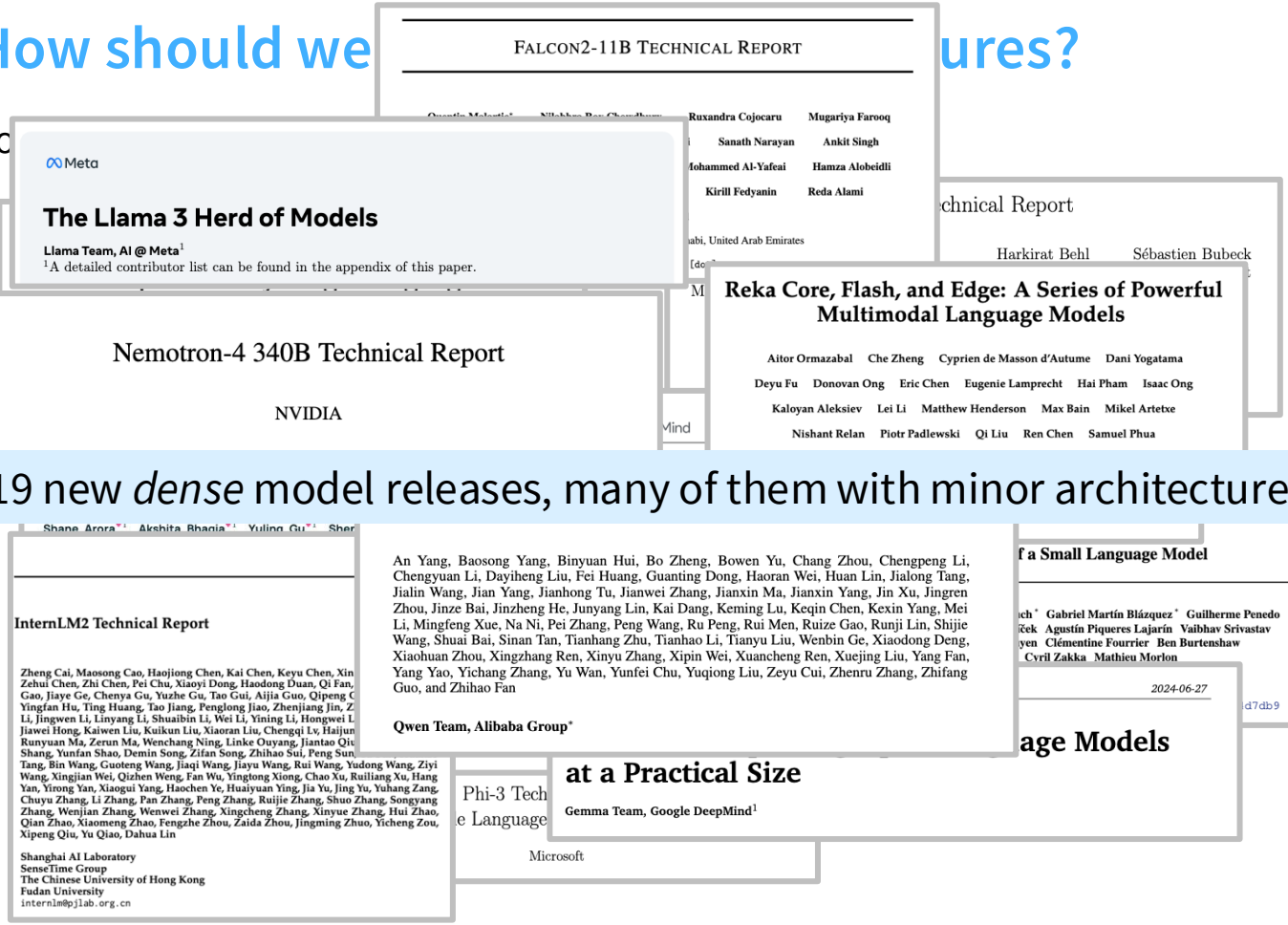
四个改动分别解决什么

Pre-Norm 改善深层残差路径；RMSNorm 简化归一化；RoPE 作用在每层注意力的 Q/K 上；SwiGLU 给逐 token FFN 增加乘法门控。去 bias 是另一项常用选择，并非 RoPE 或 Pre-Norm 的数学要求。现代模型仍保留注意力与 FFN 两种互补作用：前者在 token 之间交换信息，后者对每个 token 的通道做相同形式的非线性变换。这里的“FFN”不是连接整个序列的大型全连接层。

本节参考（完整书目见讲义末尾）：[slides] · [transformer]

How should we [illegible] ures?

Lo[illegible]



FALCON2-11B TECHNICAL REPORT

Ruxandra Cojocaru Mugariya Farooq

Sanath Narayan Ankit Singh

Mohammed Al-Yafeai Hamza Alobeidli

Kirill Fedyanin Reda Alami

abi, United Arab Emirates

Meta

The Llama 3 Herd of Models

Llama Team, AI @ Meta[1]

[1]A detailed contributor list can be found in the appendix of this paper.

chnical Report

Harkirat Behl Sébastien Bubeck

Nemotron-4 340B Technical Report

NVIDIA

Reka Core, Flash, and Edge: A Series of Powerful Multimodal Language Models

Aitor Ormazabal Che Zheng Cyprien de Masson d'Autume Dani Yogatama

Deyu Fu Donovan Ong Eric Chen Eugenie Lamprecht Hai Pham Isaac Ong

Kaloyan Aleksiev Lei Li Matthew Henderson Max Bain Mikel Artetxe

Nishant Relan Piotr Padlewski Qi Liu Ren Chen Samuel Phua

Mind

Over 19 new *dense* model releases, many of them with minor architecture tweaks..

InternLM2 Technical Report

Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin
Zehui Chen, Zhi Chen, Pei Chu, Xiaoyi Dong, Haodong Duan, Qi Fan,
Gao, Jiaye Ge, Chenya Gu, Yuzhe Gu, Tao Gui, Aijia Guo, Qipeng G
Yingfan Hu, Ting Huang, Tao Jiang, Penglong Jiao, Zhenjiang Jin, Z
Li, Jingwen Li, Linyang Li, Shuaibin Li, Wei Li, Yining Li, Hongwei L
Jiawei Hong, Kaiwen Liu, Kuikun Liu, Xiaoran Liu, Chengqi Lv, Haijun
Runyuan Ma, Zerun Ma, Wenchang Ning, Linke Ouyang, Jiantao Qiu
Shang, Yunfan Shao, Demin Song, Zifan Song, Zhihao Sui, Peng Sun
Tang, Bin Wang, Guoteng Wang, Jiaqi Wang, Jiayu Wang, Rui Wang, Yudong Wang, Ziyi
Wang, Xingjian Wei, Qizhen Weng, Fan Wu, Yingtong Xiong, Chao Xu, Ruiliang Xu, Hang
Yan, Yirong Yan, Xiaogui Yang, Haochen Ye, Huaiyuan Ying, Jia Yu, Jing Yu, Yuhang Zang,
Chuyu Zhang, Li Zhang, Pan Zhang, Peng Zhang, Ruijie Zhang, Shuo Zhang, Songyang
Zhang, Wenjian Zhang, Wenwei Zhang, Xingcheng Zhang, Xinyue Zhang, Hui Zhao,
Qian Zhao, Xiaomeng Zhao, Fengzhe Zhou, Zaida Zhou, Jingming Zhuo, Yicheng Zou,
Xipeng Qiu, Yu Qiao, Dahua Lin

Shanghai AI Laboratory
SenseTime Group
The Chinese University of Hong Kong
Fudan University
internlm@pjlab.org.cn

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan

Qwen Team, Alibaba Group*

f a Small Language Model

Gabriel Martín Blázquez* Guilherme Penedo

Agustín Piqueres Lajarín Vaibhav Srivastav

Clémentine Fourrier Ben Burtenshaw

Cyril Zakka Mathieu Morlon

2024-06-27

d7db9

age Models

at a Practical Size

Gemma Team, Google DeepMind[1]

Phi-3 Tech

e Language

Microsoft

August 5, 2025 Release Product

Introducing gpt-oss

gpt-oss-120b and gpt-oss-20b push the frontier of open-weight reasoning model

Explore on Hugging Face ↗ Read model card ↗

MiniMax-M1: Scaling Test-Time Efficiently with Lightning Attent

MiniMax[1]

Llama 4:
Leading Multimodal In

Llama 4 Behemoth

Llama 4 Maverick

Llama 4 Scout

deepseek

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

DeepSeek-AI

research@deepseek.com

2025-05-15

efficien
hybrid models for
enterprise

Published 02 October 2025

modal,

Liquid

Try LFM • Docs • LEAP • Disco

MiniMax M2 & Agent: Ingenious in Simplicity

Access API → Coding Plan → Try Agent Now

NVIDIA

NVIDIA Nemotron 3: Efficient and

models—Nano, Super, and Ultra. These models deliver strong
apabilities. The Nemotron 3 family uses a Mixture-of-Experts
re to provide best-in-class throughput and context lengths of
dels are trained with NVFP4 and incorporate LatentMoE, a
lity. The two larger models also include MTP layers for faster

INTERN-S1: A SCIENTIFIC MULTIMODAL FOUNDATION MODEL

Intern-S1 Team, Shanghai AI Laboratory

foundation models—together. We're
ratch, and sharing and programmatically
e, the data, the experiments, the
ite anyone who shares our vision of open
nd contribute, whether you want to try out
ithm, dataset, evaluation…there is a lot to

ERNIE 4.5 Technical Repo

KIMI K2: OPEN AGENTIC INTELLIGENCE

TECHNICAL REPORT OF KIMI K2

Kimi Team

oundation models
lixture-of-Experts
taining 424B total
MoE architecture,

of Maryland
here.

2025-12-22 · Research

GLM-4.7: Advancing the Coding Capability

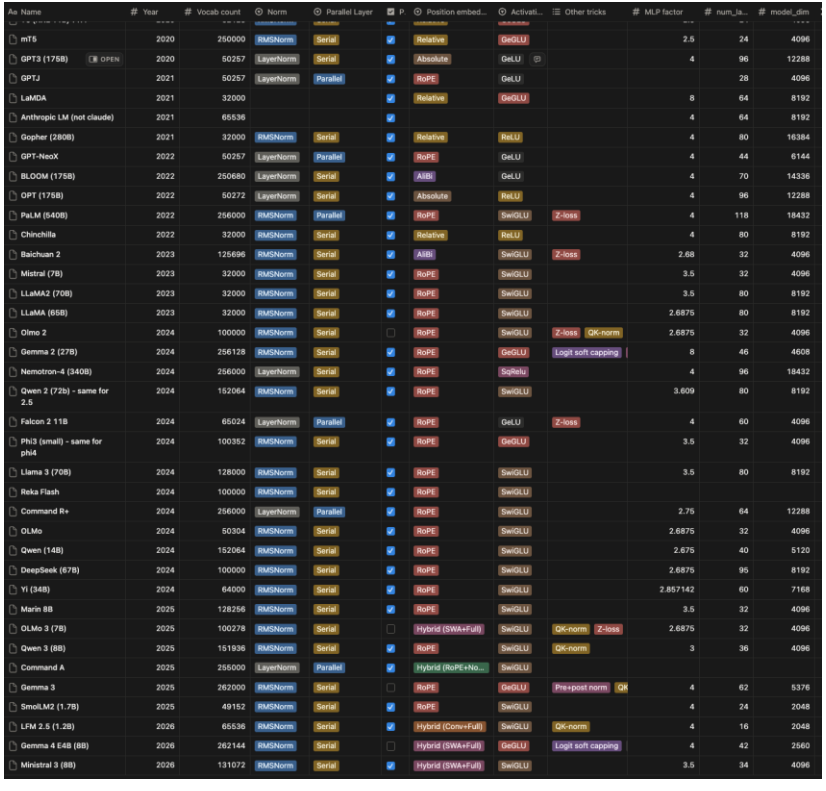
Try it at Z.ai ↗ Call it at Z.ai ↗ Z.ai Coding Plan ↗ GitHub ↗ HuggingFace ↗ Tech Report ↗

Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding

StepFun Inc.

Let’s look at the data (on dense architectures)

Learn from the many other models (and papers) out there



Name	Year	Vocab count	Norm	Parallel Layer	P	Position embed...	Activati...	Other tricks	MLP factor	num_la...	model_dim
mT5	2020	250000	RMSNorm	Serial	☑	Relative	GeGLU		2.5	24	4096
GPT3 (175B) OPEN	2020	50257	LayerNorm	Serial	☑	Absolute	GeLU		4	96	12288
GPTJ	2021	50257	LayerNorm	Parallel	☑	RoPE	GeLU			28	4096
LaMDA	2021	32000			☑	Relative	GeGLU		8	64	8192
Anthropic LM (not claude)	2021	65536			☑				4	64	8192
Gopher (280B)	2021	32000	RMSNorm	Serial	☑	Relative	ReLU		4	80	16384
GPT-NeoX	2022	50257	LayerNorm	Parallel	☑	RoPE	GeLU		4	44	6144
BLOOM (175B)	2022	250680	LayerNorm	Serial	☑	AliBi	GeLU		4	70	14336
OPT (175B)	2022	50272	LayerNorm	Serial	☑	Absolute	ReLU		4	96	12288
PaLM (540B)	2022	256000	RMSNorm	Parallel	☑	RoPE	SwiGLU	Z-loss	4	118	18432
Chinchilla	2022	32000	RMSNorm	Serial	☑	Relative	ReLU		4	80	8192
Baichuan 2	2023	125696	RMSNorm	Serial	☑	AliBi	SwiGLU	Z-loss	2.68	32	4096
Mistral (7B)	2023	32000	RMSNorm	Serial	☑	RoPE	SwiGLU		3.5	32	4096
LLaMA2 (70B)	2023	32000	RMSNorm	Serial	☑	RoPE	SwiGLU		3.5	80	8192
LLaMA (65B)	2023	32000	RMSNorm	Serial	☑	RoPE	SwiGLU		2.6875	80	8192
Olmo 2	2024	100000	RMSNorm	Serial	☐	RoPE	SwiGLU	Z-loss QK-norm	2.6875	32	4096
Gemma 2 (27B)	2024	256128	RMSNorm	Serial	☑	RoPE	GeGLU	Logit soft capping	8	46	4608
Nemotron-4 (340B)	2024	256000	LayerNorm	Serial	☑	RoPE	SqReLu		4	96	18432
Qwen 2 (72b) - same for 2.5	2024	152064	RMSNorm	Serial	☑	RoPE	SwiGLU		3.609	80	8192
Falcon 2 11B	2024	65024	LayerNorm	Parallel	☑	RoPE	GeLU	Z-loss	4	60	4096
Phi3 (small) - same for phi4	2024	100352	RMSNorm	Serial	☑	RoPE	GeGLU		3.5	32	4096
Llama 3 (70B)	2024	128000	RMSNorm	Serial	☑	RoPE	SwiGLU		3.5	80	8192
Reka Flash	2024	100000	RMSNorm	Serial	☑	RoPE	SwiGLU				
Command R+	2024	256000	LayerNorm	Parallel	☑	RoPE	SwiGLU		2.75	64	12288
OLMo	2024	50304	RMSNorm	Serial	☑	RoPE	SwiGLU		2.6875	32	4096
Qwen (14B)	2024	152064	RMSNorm	Serial	☑	RoPE	SwiGLU		2.675	40	5120
DeepSeek (67B)	2024	100000	RMSNorm	Serial	☑	RoPE	SwiGLU		2.6875	95	8192
Yi (34B)	2024	64000	RMSNorm	Serial	☑	RoPE	SwiGLU		2.857142	60	7168
Marin 8B	2025	128256	RMSNorm	Serial	☑	RoPE	SwiGLU		3.5	32	4096
OLMo 3 (7B)	2025	100278	RMSNorm	Serial	☐	Hybrid (SWA+Full)	SwiGLU	QK-norm Z-loss	2.6875	32	4096
Qwen 3 (8B)	2025	151936	RMSNorm	Serial	☑	RoPE	SwiGLU	QK-norm	3	36	4096
Command A	2025	255000	LayerNorm	Parallel	☑	Hybrid (RoPE+No...	SwiGLU				
Gemma 3	2025	262000	RMSNorm	Serial	☐	RoPE	GeGLU	Pre+post norm QK	4	62	5376
SmolLM2 (1.7B)	2025	49152	RMSNorm	Serial	☑	RoPE	SwiGLU		4	24	2048
LFM 2.5 (1.2B)	2026	65536	RMSNorm	Serial	☑	Hybrid (Conv+Full)	SwiGLU	QK-norm	4	16	2048
Gemma 4 E4B (8B)	2026	262144	RMSNorm	Serial	☐	Hybrid (SWA+Full)	GeGLU	Logit soft capping	4	42	2560
Ministral 3 (8B)	2026	131072	RMSNorm	Serial	☑	Hybrid (SWA+Full)	SwiGLU		3.5	34	4096

We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

第 3 讲 · 原页 5–7 · 中文讲解

怎样读模型架构调查表

图中模型名单不是排行榜

这三页把不同年份和团队的论文封面、模型配置表摆在一起，说明设计空间很大。深色表格的每一行代表一个模型，每一列代表一个设计选择；纵向重复的颜色揭示共享惯例，少量异色行提示例外。读者不必记住全部模型名称，而要学会问：是不是同一架构谱系沿用了默认值？是否有公开消融支撑？是否因为硬件、上下文长度或部署需求而偏离默认值？

编者补充：经验共识与因果证据

若20个模型中18个采用一种归一化，这不是18次随机对照实验。团队之间可能继承配置，且训练数据、优化器、规模一起改变。能支持因果解释的证据更接近：固定数据、训练 token 数与预算，只替换一种设计，并报告多个任务和稳定性。第7页强调 dense 架构调查；含 MoE 的发布封面并不意味着后续所有参数比较也包含路由专家。本文后续的参数估算默认稠密块，不能直接用来计算 MoE 的总参数与激活参数。

本节参考（完整书目见讲义末尾）：[slides]

What are we going to cover?

Common architecture variations

- Activations, FFN
- Attention variants
- Position embeddings

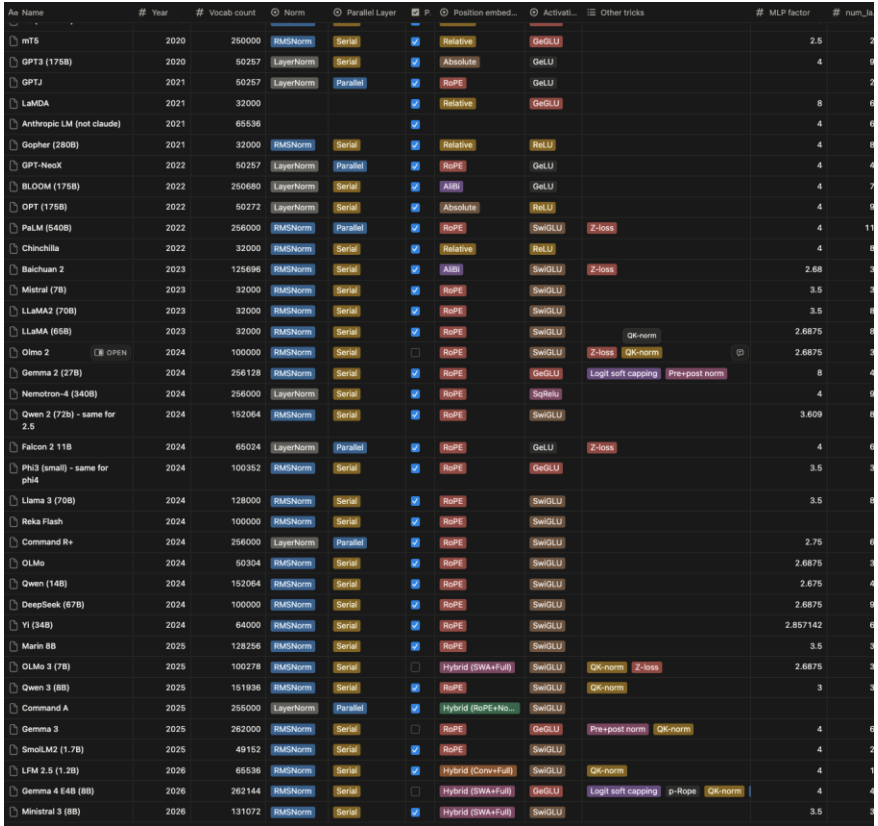
Hyperparameters that (do or don’t) matter

- What is ff_dim? Do multi_head dims always sum to model_dim?
- How many vocab elements?

Stability tricks

Architecture variations..

Let’s think about the core architecture piece



Name	Year	Vocab count	Norm	Parallel Layer	P.	Position embed...	Activati...	Other tricks	MLP factor	num_la
mT5	2020	250000	RMSNorm	Serial	✓	Relative	GeGLU		2.5	
GPT3 (175B)	2020	50257	LayerNorm	Serial	✓	Absolute	GeLU		4	
GPTJ	2021	50257	LayerNorm	Parallel	✓	RoPE	GeLU			
LaMDA	2021	32000			✓	Relative	GeGLU		8	
Anthropic LM (not claude)	2021	65536			✓				4	
Gopher (280B)	2021	32000	RMSNorm	Serial	✓	Relative	ReLU		4	
GPT-NeoX	2022	50257	LayerNorm	Parallel	✓	RoPE	GeLU		4	
BLOOM (175B)	2022	250680	LayerNorm	Serial	✓	ALiBi	GeLU		4	
OPT (175B)	2022	50272	LayerNorm	Serial	✓	Absolute	ReLU		4	
PaLM (540B)	2022	256000	RMSNorm	Parallel	✓	RoPE	SwiGLU	Z-loss	4	
Chinchilla	2022	32000	RMSNorm	Serial	✓	Relative	ReLU		4	
Baichuan 2	2023	125696	RMSNorm	Serial	✓	ALiBi	SwiGLU	Z-loss	2.68	
Mistral (7B)	2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	
LLaMA2 (70B)	2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	
LLaMA (65B)	2023	32000	RMSNorm	Serial	✓	RoPE	SwiGLU	QK-norm	2.6875	
Olmo 2 OPEN	2024	100000	RMSNorm	Serial		RoPE	SwiGLU	Z-loss QK-norm	2.6875	
Gemma 2 (27B)	2024	256128	RMSNorm	Serial	✓	RoPE	GeGLU	Logit soft capping Pre+post norm	8	
Nemotron-4 (340B)	2024	256000	LayerNorm	Serial	✓	RoPE	SqRelu		4	
Qwen 2 (72b) - same for 2.5	2024	152064	RMSNorm	Serial	✓	RoPE	SwiGLU		3.609	
Falcon 2 11B	2024	65024	LayerNorm	Parallel	✓	RoPE	GeLU	Z-loss	4	
Phi3 (small) - same for phi4	2024	100352	RMSNorm	Serial	✓	RoPE	GeGLU		3.5	
Llama 3 (70B)	2024	128000	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	
Reka Flash	2024	100000	RMSNorm	Serial	✓	RoPE	SwiGLU			
Command R+	2024	256000	LayerNorm	Parallel	✓	RoPE	SwiGLU		2.75	
OLMo	2024	50304	RMSNorm	Serial	✓	RoPE	SwiGLU		2.6875	
Qwen (14B)	2024	152064	RMSNorm	Serial	✓	RoPE	SwiGLU		2.675	
DeepSeek (67B)	2024	100000	RMSNorm	Serial	✓	RoPE	SwiGLU		2.6875	
Yi (34B)	2024	64000	RMSNorm	Serial	✓	RoPE	SwiGLU		2.857142	
Marin 8B	2025	128256	RMSNorm	Serial	✓	RoPE	SwiGLU		3.5	
OLMo 3 (7B)	2025	100278	RMSNorm	Serial		Hybrid (SWA+Full)	SwiGLU	QK-norm Z-loss	2.6875	
Qwen 3 (8B)	2025	151936	RMSNorm	Serial	✓	RoPE	SwiGLU	QK-norm	3	
Command A	2025	255000	LayerNorm	Parallel	✓	Hybrid (RoPE+No...	SwiGLU			
Gemma 3	2025	262000	RMSNorm	Serial		RoPE	GeGLU	Pre+post norm QK-norm	4	
SmolLM2 (1.7B)	2025	49152	RMSNorm	Serial	✓	RoPE	SwiGLU		4	
LFM 2.5 (1.2B)	2026	65536	RMSNorm	Serial	✓	Hybrid (Conv+Full)	SwiGLU	QK-norm	4	
Gemma 4 E4B (8B)	2026	262144	RMSNorm	Serial		Hybrid (SWA+Full)	GeGLU	Logit soft capping p-Rope QK-norm	4	
Ministral 3 (8B)	2026	131072	RMSNorm	Serial	✓	Hybrid (SWA+Full)	SwiGLU		3.5	

High level view:

- Dominance of ‘LLaMA-like’ architectures
- Trends over the years (QK-norm, Hybrid attention)

第 3 讲 · 原页 8–9 · 中文讲解

建立覆盖清单：从结构到训练，再到部署

本讲的三个层次

架构决定计算图，例如 FFN 是否门控、位置如何进入 Q/K；超参数决定图的形状，例如 d、m、h、k、L 和词表大小 V；稳定性策略影响数值与优化，例如 QK-Norm、z-loss 和 soft-capping。它们会互相影响：门控改变 FFN 参数计数，GQA 改变 K/V 投影维度，RoPE 要求旋转子空间维数为偶数。将一个配置字段孤立地复制到另一个模型，可能破坏这些关系。

本讲调查范围的边界

第9页的总体趋势是 LLaMA-like 的残差块结构广泛采用，同时 QK-Norm 与混合注意力在样本中增加。这是阅读整张表的概括，不是每个模型都相同。以下先学主干的 norm 与 FFN，再推导 RoPE，之后讨论比例选择与稳定性，最后分析 KV cache 和局部注意力；下一讲才系统讨论注意力替代方案与 MoE。

本节参考（完整书目见讲义末尾）：[slides]

Pre-vs-post norm

The one thing *everyone* agrees on (in 2024)

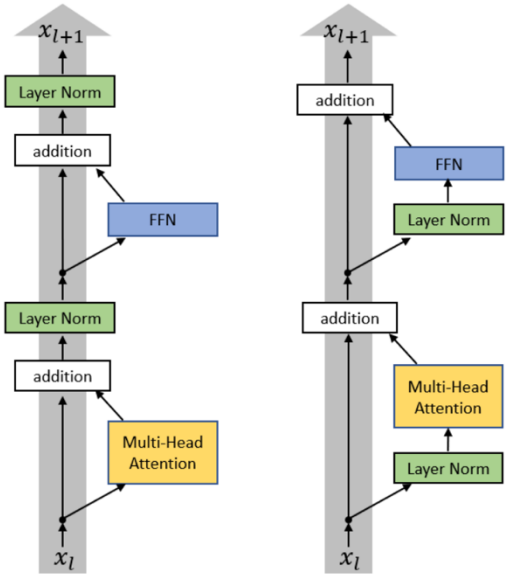


Figure from Xiong 2020

Post-LN Transformer	Pre-LN Transformer
$x_{l,i}^{post,1} = \text{MultiHeadAtt}(x_{l,i}^{post}, [x_{l,1}^{post}, \cdots, x_{l,n}^{post}])$	$x_{l,i}^{pre,1} = \text{LayerNorm}(x_{l,i}^{pre})$
$x_{l,i}^{post,2} = x_{l,i}^{post} + x_{l,i}^{post,1}$	$x_{l,i}^{pre,2} = \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \cdots, x_{l,n}^{pre,1}])$
$x_{l,i}^{post,3} = \text{LayerNorm}(x_{l,i}^{post,2})$	$x_{l,i}^{pre,3} = x_{l,i}^{pre} + x_{l,i}^{pre,2}$
$x_{l,i}^{post,4} = \text{ReLU}(x_{l,i}^{post,3} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$	$x_{l,i}^{pre,4} = \text{LayerNorm}(x_{l,i}^{pre,3})$
$x_{l,i}^{post,5} = x_{l,i}^{post,3} + x_{l,i}^{post,4}$	$x_{l,i}^{pre,5} = \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$
$x_{l+1,i}^{post} = \text{LayerNorm}(x_{l,i}^{post,5})$	$x_{l+1,i}^{pre} = x_{l,i}^{pre,5} + x_{l,i}^{pre,3}$
	Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$

Set up LayerNorm so that it doesn't affect the main residual signal path (on the left)

Almost all modern LMs use pre-norm (but BERT was post-norm)

(One somewhat funny exception – OPT350M. I don't know why this is post-norm)

第 3 讲 · 原页 10–10 · 中文讲解

Pre-Norm 与 Post-Norm：归一化放在哪里

概念与背景

残差连接的想法是让新层学习“对已有表示的修正”，而不是每层都重建全部信息。设 F 是注意力或 FFN，N 是归一化。Post-Norm 写作 y = N(x + F(x))；Pre-Norm 写作 y = x + F(N(x))。名称是相对 F 的位置而言，最重要的是看原图左侧的跨层直通线：Pre-Norm 的这条线不穿过 N。实际块有注意力和 FFN 两个子层，要分别判断。

编者补充：链式法则给出的直觉

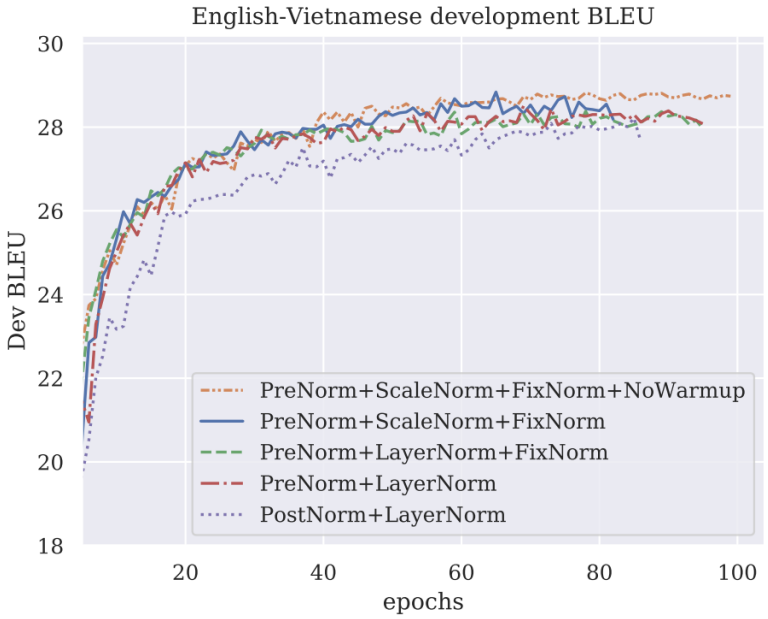
把向量对向量的导数记为 Jacobian J。Pre-Norm 有 J_y = I + J_F J_N，反向传播中保留一条乘以 I 的路径；Post-Norm 是 J_y = J_N(I + J_F)，即便沿残差分支也会经过归一化的导数。堆叠很多层时，后者更容易让梯度尺度依赖一串 J_N。这个推导说明路径结构，却不证明 Pre-Norm 永不梯度爆炸或一定达到更低最终损失：各分支相加仍可能放大、抵消，训练数据和学习率仍很重要。

实现时最容易遗漏的两点

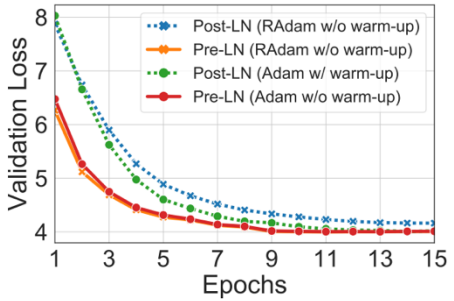
Pre-Norm 模型往往在所有块之后再加一次最终 norm，以控制进入输出头的尺度；这与在每个残差加法之后做 Post-Norm 是不同位置。归一化通常沿单个 token 的 d 个通道计算，不跨 batch，也不跨序列。第10页提到的 OPT-350M 是调查中的例外，不能据一个例外推断归一化位置无关紧要。

本节参考（完整书目见讲义末尾）：[slides] · [prenorm]

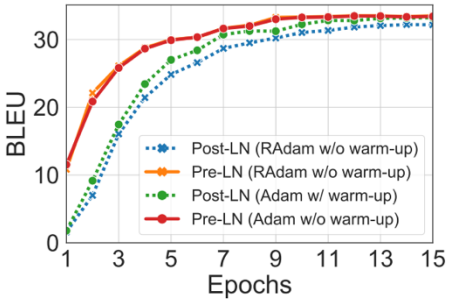
Pre-vs-post-norm, the data



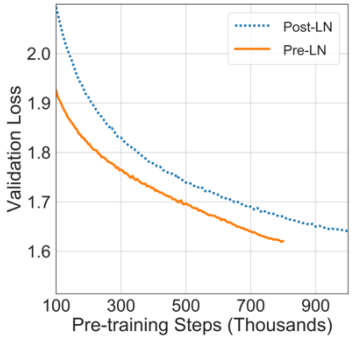
Salazar and Ngyuen 2019



(a) Validation Loss (IWSLT)



(b) BLEU (IWSLT)

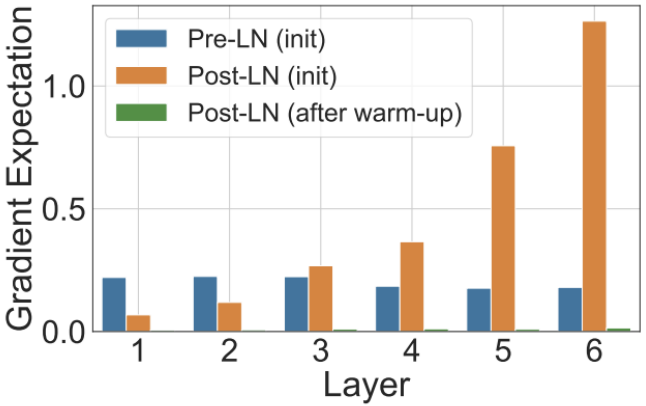


(a) Validation Loss on BERT

Figure from Xiong 2020

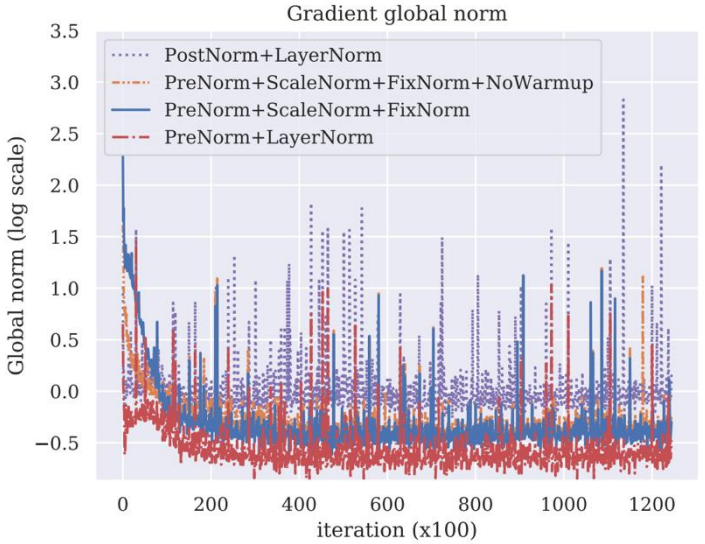
Pre-vs-post norm, explanations?

Gradient attenuation [Xiong 2020]



(a) W^1 in the FFN sub-layers

Gradient spikes [Salazar and Ngyuen]



Original stated advantage– removing warmup.
Today – stability and larger LRs for large networks

第 3 讲 · 原页 11–12 · 中文讲解

从训练曲线理解 Pre-Norm 的证据

逐图阅读

第11页左图横轴是 epoch、纵轴是翻译 BLEU，多个 norm/warmup 配置达到相近质量，但训练路径不同；右侧图将验证损失与 BLEU 分开，提醒我们稳定训练和最终质量是两个评价维度。第12页左图比较不同层的梯度期望，初始化时 Post-LN 靠近输出的层梯度更大；右图显示梯度范数随步数的尖峰。它们共同支持“归一化位置会改变优化行为”，不能把一条曲线的趋势提升为所有规模的定理。

编者补充：**warmup** 如何与梯度相连

一次简化 SGD 更新是 $\Delta\theta = -\eta g$。若初始化时某些层的梯度 g 很大，直接用目标学习率 η 会产生很大的参数跳跃；warmup 从较小 η 开始，给参数与优化器状态适应的机会。Xiong 等人的分析有初始化和均值场假设，实验说明某些 Pre-LN 配置能不依赖 warmup。今天的大模型仍可能保留 warmup，因为优化器统计、低精度和分布式批量也有启动问题；“采用 Pre-Norm”不等于“应删除 warmup”。

判断结论的正确范围

课件说现代用途更偏向稳定性与允许较大学习率。实践上应同时记录训练/验证损失、梯度范数、更新范数和异常步数。单看最终 BLEU 接近，可能忽略失败的训练尝试；单看梯度平稳，也不能证明泛化更好。本讲是在提供可检验的优化解释，不是在为每个模型指定唯一配置。

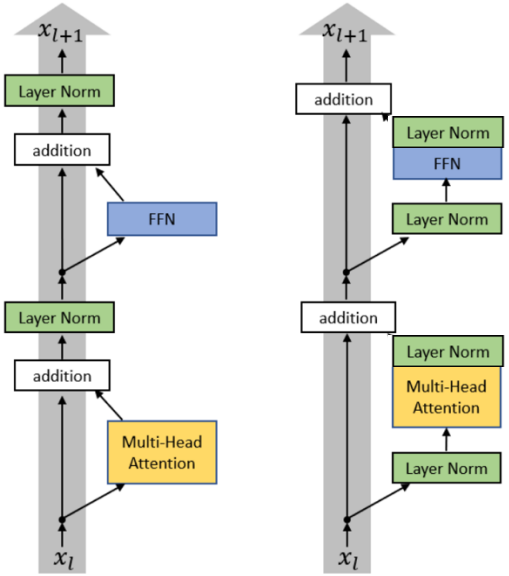
课堂补充 · **00:09:42.950–00:10:11.310**

第三方录像字幕转述：讲者回顾早期 Pre-Norm 研究希望去掉 warmup 的动机，同时明确提到现代 Transformer 训练仍做 warmup。此处支持的是把早期研究动机与今天的训练配方分开，不能写成“用了 Pre-Norm 就不需要预热”。

本节参考（完整书目见讲义末尾）：[slides] · [prenorm] · [captions]

New things – ‘double’ norm or non-residual postnorm

If putting LayerNorms in residual streams is bad.. Why not post-norm outside the stream?



Recent models: Grok, Gemma 2. Olmo 2 *only* does non-residual post norm

第 3 讲 · 原页 13–13 · 中文讲解

双归一化与分支输出归一化

沿着残差线辨认结构

第13页追问：既然把 norm 放到主残差线上会改变直通路径，能否在支路内部再加一个 norm？可以。编者用 $y = x + N_2(F(N_1(x)))$ 表示一种双 norm 形式，用 $y = x + N_2(F(x))$ 表示一种仅在分支输出归一化的形式。两者都保留外部的 $x \to y$ 直连，但内部尺度控制不同。模型具体实现可能还包含缩放参数，应以其计算图为准。

为什么“post”一词会误导

传统 Post-Norm 是 $N(x + F(x))$，而 non-residual post-norm 是 $x + N(F(x))$；归一化和加法不能交换次序。前者约束整个新状态，后者约束本层写回残差流的增量。若 $F(x)$ 的幅值很大，后者可能控制单层更新；不过同时增加一次读写、归约和可学习尺度。课件举 Grok、Gemma 2、OLMo 2 说明设计在演化，不能把它们都简化成同一张图。读论文时先写方程，再讨论名称。

本节参考（完整书目见讲义末尾）：[slides] · [gemma2]

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Notable models:

GPT3/2/1, OPT, GPT-J, BLOOM

Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{||x||_2^2 + \varepsilon}} * \gamma$$

Notable models:

LLaMA-family, PaLM, Chinchilla, T5

第 3 讲 · 原页 14–14 · 中文讲解

LayerNorm 与 RMSNorm 的精确定义

编者补充：把省略的维度写出来

对单个 token 的向量 $x \in R^d$，$\mu = (1/d)\Sigma_i x_i$，$v = (1/d)\Sigma_i (x_i-\mu)^2$。LayerNorm 的第 i 维为 $\gamma_i(x_i-\mu)/\sqrt{(v+\varepsilon)}+\beta_i$。RMSNorm 则为 $\gamma_i x_i/\sqrt{((1/d)\Sigma_j x_j^2+\varepsilon)}$。γ 是逐通道可学习缩放，β 是偏置，ε 是防止零分母的小正数。这里的方差通常用除以 d 的总体方差，不是统计估计中除以 d−1 的无偏样本方差。

手算例子与不变性

令 x=(1,3)，γ=(1,1)，β=0，暂忽略 ε。LayerNorm 的 μ=2、v=1，输出 (−1,1)；RMSNorm 的均方为 5，输出约 (0.4472,1.3416)，均值不为零。把 x 全体加10，LayerNorm 输出不变，RMSNorm 一般改变；将 x 乘正数 c，在 ε 可忽略时二者都会抵消这个整体尺度。RMSNorm 放弃平移不变性，但保留对整体正缩放的近似不变性。加上 γ 后，输出均方也不一定仍为1。

原稿公式阅读提示

第14页图中分母显示为 $\sqrt{(||x||_2^2+\varepsilon)}$，按通常欧氏范数定义缺少 1/d；标准 RMSNorm 应按上一段的均方公式理解。若某处把 $||x||_2$ 定义成均方根则另当别论，但本页没有给出这种定义。不要据该简写实现成单位二范数归一化：忽略 ε 时，那会使输出比标准 RMSNorm 小 $\sqrt{d}$ 倍。原始 PDF 保留不改，此处明确数学约定。

本节参考（完整书目见讲义末尾）：[slides] · [rmsnorm]

Why RMSNorm?

Modern explanation – it's faster (and just as good).

- **Fewer operations** (no mean calculation)
- **Fewer parameters** (no bias term to store)

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Does this explanation make sense?

Operator class	% flop
△ Tensor contraction	99.80
□ Stat. normalization	0.17
○ Element-wise	0.03

Matrix multiplies are the *vast* majority of FLOPs (and memory)

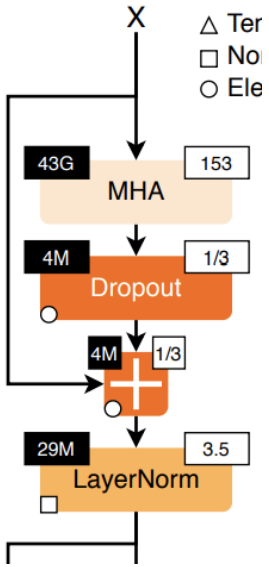
[Ivanov et al 2023]

Why RMSNorm (2)

Important lesson: FLOPS are not runtime! (we will discuss this in far more detail later)

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

RMSNorm can still matter due to the importance of *data movement*



Left top (”43G”) is FLOPS
Right top (“153”) is the FLOP-to-memory ratio

[Ivanov et al 2023]

RMSNorm - validation

RMSNorm runtime (and surprisingly, perf) gains have been seen in papers

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ	WMT EnDe
Vanilla Transformer	$223M$	$11.1T$	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02	26.62
RMS Norm	$223M$	$11.1T$	3.68	2.167 ± 0.008	**1.821**	**75.45**	**17.94**	**24.07**	**27.14**
Rezero	$223M$	$11.1T$	3.51	2.262 ± 0.003	1.939	61.69	15.64	20.90	26.37
Rezero + LayerNorm	$223M$	$11.1T$	3.26	2.223 ± 0.006	1.858	70.42	17.58	23.02	26.29
Rezero + RMS Norm	$223M$	$11.1T$	3.34	2.221 ± 0.009	1.875	70.33	17.32	23.02	26.19
Fixup	$223M$	$11.1T$	2.95	2.382 ± 0.012	2.067	58.56	14.42	23.02	26.31

Narang et al 2020

第 3 讲 · 原页 15–17 · 中文讲解

为什么减少一点归一化计算仍可能变快

从图表看 **FLOPs** 与用时的差别

第15页表格中张量收缩约占99.80%的 FLOPs，统计归一化只占0.17%；第16页却显示该测量中归一化占约25.5%的运行时间，逐元素操作也占13.5%。这并不矛盾：矩阵乘可复用输入并饱和计算单元，归约和逐元素算子常要搬运大量数据、等待同步。这里是被引用实现与硬件上的剖析，不是任意 Transformer 的固定比例。

编者补充：用 **roofline** 思想估算

理想化下界 $T \geq \max(F/P, B/BW)$，F 为浮点操作数，P 为峰值计算吞吐，B 为实际搬运字节，BW 为带宽。算术强度 $I=F/B$ 很小时，减少 F 不一定减少瓶颈时间，减少中间激活读写、kernel launch 或融合算子可能更有效。假设某步骤计算只需1微秒，而读写要8微秒，计算量减半仍受约8微秒约束；若融合后读写减半，才可能接近4微秒。这是教学估算，不是测量结果。

RMSNorm 的收益来自哪里

RMSNorm 省去求均值与减均值，通常也无 β，可能简化归约与内存访问。γ/β 本身相对大矩阵很小，不能把全部加速归因于“少保存 d 个参数”；是否使用融合 kernel、归约实现和精度策略同样关键。第17页的对照表比较模型质量与 step/s，提示要同时测性能与效果。RMSNorm 常能维持质量，但表中的结果有任务和训练设置边界，不构成处处等价于 LayerNorm 的证明。

课堂补充 · **00:15:34.300–00:16:46.530**

第三方录像字幕转述：讲者强调0.17%的 FLOPs 也可能对应约25%的运行时间，并限定这取决于 workload 与实现；图中白标签表示算术强度，黑标签表示 FLOPs。数据搬运的重要性连接了第2讲的成本分析与本讲的 norm 选择。

本节参考（完整书目见讲义末尾）：[slides] · [rmsnorm] · [captions] · [data_movement] · [narang]

More generally: dropping bias terms

Most modern transformers don't have bias terms.

Original Transformer:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Most implementations (if they're not gated):

$$FFN(x) = \sigma(xW_1)W_2$$

Reasons: memory (similar to RMSnorm) and optimization stability

LayerNorm: recap

- Basically everyone does non-residual norm (often prenorm)
  - Intuition – keep the good parts of residual connections
  - Observations – nicer gradient propagation, fewer spike
  - Some people add a second norm outside the residual stream

- Most people do RMSnorm
  - In practice, works as well as LayerNorm
  - But, has fewer parameters to move around, which saves on wallclock time
  - People more generally drop bias terms since the compute/param tradeoffs are not great.

第 3 讲 · 原页 18–19 · 中文讲解

去掉 bias 与归一化小结

编者补充：偏置是什么，省了多少

普通 FFN 写作 $\sigma(xW_1+b_1)W_2+b_2$，其中 $W_1 \in R^{(d\times m)}$、$W_2 \in R^{(m\times d)}$，bias 共 m+d 个数，而矩阵有2dm 个数。若 d=1024、m=4096，矩阵有8,388,608个参数，bias 只有5,120个，约为矩阵参数的0.061%。因此省模型体积不是唯一动机；简化 kernel、少一次广播加法，以及训练经验中的稳定性更值得关注。

不能错误地“证明 **bias** 没用”

下一层有 norm 并不意味着所有 bias 都被消掉。LayerNorm 只能抵消沿所有通道相同的整体平移，一般的逐通道 bias 仍会改变向量；RMSNorm 连整体平移不变性也没有。激活前的 bias 还会改变门何时打开。因此去 bias 是改变函数族的工程选择，只是很多大模型能够在这一限制下很好地训练。小任务、其他网络或特定预训练权重不应机械删除 bias。

读完这一组应能区分的两个轴

归一化“放在哪里”与“用哪一种”是两个独立问题：Pre/Post/分支输出决定计算图，LayerNorm/RMSNorm 决定统计量。现代经验常把保留残差直通线、RMSNorm 和无 bias 组合起来。若训练异常，先确认实现轴、ε、dtype 和残差次序，再考虑换一种名称；仅把 LayerNorm 换成 RMSNorm 并不能修复把归一化误放在残差主线上的错误。

本节参考（完整书目见讲义末尾）：[slides]

Activations

A whole zoo of activations ..

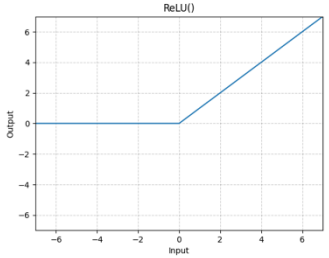
ReLU, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, SeLU, SwiGLU, LiGLU

What are these things? What do people use? Does it matter?

A few of the common activations

ReLU

$$FF(x) = \max(0, xW_1)\, W_2$$

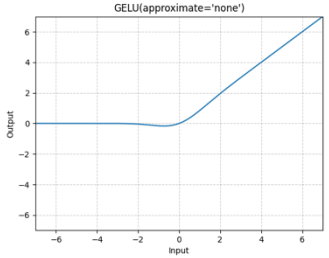


Notable models:

Original transformer, T5, Gopher, Chinchilla, OPT

GeLU

$$FF(x) = \mathrm{GELU}(xW_1)W_2$$
$$GELU(x) \coloneqq x\Phi(x)$$



Notable models:

GPT1/2/3, GPTJ, GPT-Neox, BLOOM

SwiGLU / GeGLU (next slide..)

Notable models:

Llama, PaLM,T5 v1.1, *most* models post 2023

第 3 讲 · 原页 20–21 · 中文讲解

激活函数：FFN 为什么需要非线性

编者补充：若没有激活会怎样

设每个 token 的行向量为 x，FFN(x)=σ(xW_1)W_2。若 σ 是恒等映射，两个矩阵可合成 W_1W_2，这个子层仍是一个线性映射，扩展到 m 维并没有增加非线性表达能力。加入激活后，模型可按输入改变哪些中间特征参与输出。注意力负责读其他 token，FFN 则在读到的表示上进行逐位置特征加工。

ReLU、GELU 与 Swish 的区别

ReLU(z)=max(0,z)，正半轴保持不变，负半轴为0。GELU(z)=zΦ(z)，Φ 是标准正态分布的累积分布函数；它连续地衰减负值，不是在每次前向传播中随机丢弃神经元。Swish 在本讲指 z·sigmoid(z)，通常也叫 SiLU；sigmoid(z)=1/(1+exp(−z))。以 z=−1 为例，ReLU 为0，GELU 约−0.1587，SiLU 约−0.2689。后两者保留小的负输出，导数也较平滑，但“更平滑”本身不是普遍更优的证明。

读原图曲线的方法

第21页 ReLU 在0处转折，GELU 在负半轴有小幅下探。它们不是概率输出层，值可以为负或大于1；不要因 Φ 和 sigmoid 是概率形状的函数，就把整个 GELU/Swish 当成门概率。课件列出的模型例子说明多个激活都能成功，接下来要比较的是含额外分支的 GLU，而不只是替换一个逐元素函数。

本节参考（完整书目见讲义末尾）：[slides]

Gated activations (*GLU)

GLUs modify the ‘first part’ of a FF layer

$$FF(x) = \max(0, xW_1)\, W_2$$

Instead of a linear + ReLU, augment the above with an (entrywise) linear term

$$\max(0, xW_1) \rightarrow \max(0, xW_1) \otimes (xV)$$

This gives the gated variant (ReGLU) – note that we have an extra parameter (V)

$$\mathrm{FF}_{\mathrm{ReGLU}}(x) = (\max(0, xW_1) \otimes xV)\, W_2$$

Gated variants of standard FF layers

GeGLU

$$\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) = (\mathrm{GELU}(xW) \otimes xV)W_2$$

Notable models:

T5 v1.1, mT5, LaMDA, Phi3, Gemma 2, Gemma 3, Gemma 4

SwiGLU (swish is $x * \mathrm{sigmoid}(x)$)

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$$

Notable models:

LLaMa 1/2/3, PaLM, Mistral, OlMo, *most models post 2023*

Note: Gated models use smaller dimensions for the d_{ff} by 2/3

第 3 讲 · 原页 22–23 · 中文讲解

门控 FFN：SwiGLU 与 GeGLU

从两块矩阵变成三块矩阵

门控 FFN 先并行投影同一个输入：a=xW，b=xV，再做逐元素乘法 σ(a)⊙b，最后乘 W_2 回到模型宽度。W、V∈R^(d×m)，W_2∈R^(m×d)。ReGLU 用 ReLU，GeGLU 用 GELU，SwiGLU 用 SiLU。原页的 ⊗ 在这里指同位置元素相乘，不是矩阵乘、外积或 Kronecker 积。门控使一个输入相关分支调节另一个分支，产生乘法交互。

编者补充：小向量与参数预算

设投影结果 a=(1,−1)，b=(2,3)，先取 ReGLU 便于手算：ReLU(a)=(1,0)，乘 b 后得到 (2,0)；若 W_2 为2×2单位阵，输出仍为(2,0)。SwiGLU 则约得到(1.4621,−0.8068)，说明门不是0/1开关，也不保证非负。普通无 bias FFN 参数为2dm，门控 FFN 为3dm。为公平比较，令3dm_g=2dm_plain，得 m_g=(2/3)m_plain。普通 m_plain=4d 时，对应 m_g=8d/3。这个2/3是预算匹配，不是网络成立条件。

实现与比较时的边界

若 d=768，普通 m=3072 有4,718,592个矩阵参数，门控 m=2048 也恰为4,718,592。真实模型可能把 m 向适合硬件的倍数取整，或主动扩大 FFN；因此“用了 SwiGLU”并不能单独决定 m。比较两种 FFN 时若都用4d，门控块会多50%矩阵参数，质量提升便不能全归因于门控结构。

课堂补充 · **00:24:31.390–00:25:40.810**

第三方录像字幕转述：将门控 FFN 的中间维度缩到原来的2/3，是为了使总参数量保持相当；讲者明确说这不是铁律。对原实验增益的评价是幅度较小但较一致，并提到重复训练与误差条。因此本节把参数匹配推导与经验质量结论分别说明。

本节参考（完整书目见讲义末尾）：[slides] · [glu] · [captions]

Do gated linear units work?

Yes, fairly consistently so.

	Score Average	CoLA MCC	SST-2 Acc
FFN_{ReLU}	83.80	51.32	94.04
FFN_{GELU}	83.86	53.48	94.04
$\text{FFN}_{\text{Swish}}$	83.60	49.79	93.69
FFN_{GLU}	84.20	49.16	94.27
$\text{FFN}_{\text{GEGLU}}$	84.12	53.65	93.92
$\text{FFN}_{\text{Bilinear}}$	83.79	51.02	**94.38**
$\text{FFN}_{\text{SwiGLU}}$	84.36	51.59	93.92
$\text{FFN}_{\text{ReGLU}}$	**84.67**	**56.16**	**94.38**
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Shazeer 2020

Do gated linear units work (2)?

Yes, with other works corroborating Shazeer 2020

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ
Vanilla Transformer	$223M$	$11.1T$	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02
GeLU	$223M$	$11.1T$	3.58	2.179 ± 0.003	1.838	**75.79**	**17.86**	**25.13**
Swish	$223M$	$11.1T$	3.62	2.186 ± 0.003	1.847	**73.77**	17.74	**24.34**
ELU	$223M$	$11.1T$	3.56	2.270 ± 0.007	1.932	67.83	16.73	23.02
GLU	$223M$	$11.1T$	3.59	2.174 ± 0.003	**1.814**	**74.20**	**17.42**	24.34
GeGLU	$223M$	$11.1T$	3.55	2.130 ± 0.006	**1.792**	**75.96**	**18.27**	**24.87**
ReGLU	$223M$	$11.1T$	3.57	2.145 ± 0.004	**1.803**	**76.17**	**18.36**	**24.87**
SeLU	$223M$	$11.1T$	3.55	2.315 ± 0.004	1.948	68.76	16.76	22.75
SwiGLU	$223M$	$11.1T$	3.53	2.127 ± 0.003	**1.789**	**76.00**	**18.20**	**24.34**
LiGLU	$223M$	$11.1T$	3.59	2.149 ± 0.005	**1.798**	**75.34**	**17.97**	**24.34**
Sigmoid	$223M$	$11.1T$	3.63	2.291 ± 0.019	1.867	**74.31**	17.51	23.02
Softplus	$223M$	$11.1T$	3.47	2.207 ± 0.011	1.850	**72.45**	17.65	**24.34**

Narang et al 2020

Gating, activations

- **Many variations (ReLU, GeLU, *GLU) across models.**

- ***GLU isn't *necessary* for a working model (see GPT3), but it's rare to see others..**

  Some outlier models..

  Nemotron 340B (Squared ReLU)

- **Evidence points towards somewhat consistent gains from Swi/GeGLU**

第 3 讲 · 原页 24–26 · 中文讲解

门控激活的实验支持与边界

逐表看增益是否一致

第24页来自 Shazeer 的实验，列含平均分、CoLA 与 SST-2 等指标；不同列最优行不完全相同，不能说某一种门控在每个任务都胜出。第25页又列出参数、操作量、step/s、早期和最终损失、下游分数。比如表中的 vanilla 最终损失1.838、SwiGLU 1.789、GeGLU 1.792，支持这些实验中门控变体改善了语言建模目标；同时 step/s 不完全一样，参数接近并不代表运行速度相同。

编者补充：如何做一个能解释的消融

至少要对齐数据、token 数、优化器与学习率计划，再选择对齐参数量、总训练 FLOPs 或实际墙钟时间中的一个预算。它们是不同问题，不能全部混称“公平”。例如若 A 在相同步数下更好却每步慢 20%，有限时间预算中应比较同总用时能训练到的质量。较小增益还需要多个随机种子或误差范围；本页提供的是已有实验的经验方向，不替代自己规模上的验证。

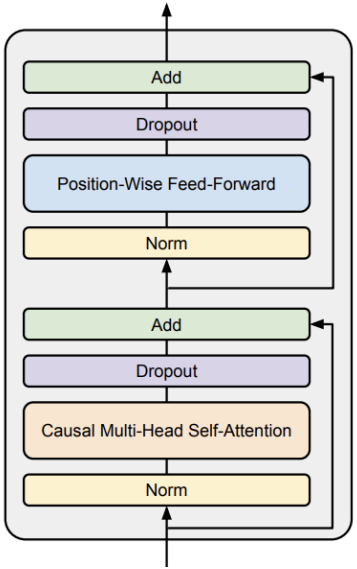
从例外理解默认值

GPT 系列用非门控激活也能训练出强模型，Nemotron 的 squared ReLU 提醒我们不存在一条“没有 GLU 就不成立”的定理。门控被广泛采用，适合作为课程和新实验的默认起点；但激活稀疏性、kernel 支持、推理吞吐等目标可能改变选择。不要仅因名字更新，就在已有预训练权重上直接替换激活。

本节参考（完整书目见讲义末尾）：[slides] · [glu] · [narang]

Serial vs Parallel layers

Normal transformer blocks are *serial* – they compute attention, then the MLP



Could we parallelize the transformer block?

Parallel layers

A few models (GPTJ, PaLM, GPT-NeoX) do parallel layers. Originally in GPT-J

> **Parallel Layers** – We use a "parallel" formulation in each Transformer block (Wang & Komatsuzaki, 2021), rather than the standard "serialized" formulation. Specifically, the standard formulation can be written as:
>
> $$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x)))$$
>
> Whereas the parallel formulation can be written as:
>
> $$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$
>
> The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused. Ablation experiments showed a small quality degradation at 8B scale but no quality degradation at 62B scale, so we extrapolated that the effect of parallel layers should be quality neutral at the 540B scale.

If implemented right, LayerNorm can be shared, and matrix multiplies can be fused

Recent Models: Cohere Command A, Falcon 2 11B, Command R+

Summary: architectures

Pre-vs-post norm:

- Everyone does non-residual norm (except OPT350M), likely with good reason.

Layer vs RMSnorm:

- RMSnorm has clear compute wins, sometimes even performance

Gating:

- GLUs are consensus now

Serial vs parallel layers:

- Most models now use serial layers

Name	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)	2019	RMSNorm	Serial	☑	Relative	ReLU
GPT2	2019	LayerNorm	Serial	☑	Absolute	GeLU
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	☑	Relative	GeGLU
mT5	2020	RMSNorm	Serial	☑	Relative	GeGLU
GPT3 (175B)	2020	LayerNorm	Serial	☑	Absolute	GeLU
GPTJ	2021	LayerNorm	Parallel	☑	RoPE	GeLU
LaMDA	2021			☑	Relative	GeGLU
Anthropic LM (not claude)	2021			☑		
Gopher (280B)	2021	RMSNorm	Serial	☑	Relative	ReLU
GPT-NeoX	2022	LayerNorm	Parallel	☑	RoPE	GeLU
BLOOM (175B)	2022	LayerNorm	Serial	☑	AliBi	GeLU
OPT (175B)	2022	LayerNorm	Serial	☑	Absolute	ReLU
PaLM (540B)	2022	RMSNorm	Parallel	☑	RoPE	SwiGLU
Chinchilla	2022	RMSNorm	Serial	☑	Relative	ReLU
Mistral (7B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA2 (70B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA (65B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
GPT4	2023			☐		
Baichuan 2	2023	RMSNorm	Serial	☑	AliBi	SwiGLU
Olmo 2	2024	RMSNorm	Serial	☐	RoPE	SwiGLU
Gemma 2 (27B)	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Nemotron-4 (340B)	2024	LayerNorm	Serial	☑	RoPE	SqRelu
Qwen 2 (72b) - same for 2.5	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Falcon 2 11B	2024	LayerNorm	Parallel	☑	RoPE	GeLU
Phi3 (small) - same for phi4	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Llama 3 (70B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Reka Flash	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Command R+	2024	LayerNorm	Parallel	☑	RoPE	SwiGLU
OLMo	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Qwen (14B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
DeepSeek (67B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Yi (34B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Mixtral of Experts	2024			☐		
Command A	2025	LayerNorm	Parallel	☑	Hybrid (RoPE+NoPE)	SwiGLU
Gemma 3	2025	RMSNorm	Serial	☐	RoPE	GeGLU
SmolLM2 (1.7B)	2025	RMSNorm	Serial	☑	RoPE	SwiGLU

第 3 讲 · 原页 27–29 · 中文讲解

串行与并行残差块

编者补充：两种数据依赖

串行块先算 $u=x+A(N_1(x))$，再算 $y=u+F(N_2(u))$，FFN 读到了本层注意力更新后的 u。并行块可写 $y=x+A(N(x))+F(N(x))$，两个分支都读同一个归一化输入。共享 N 的情况下，可将 Q/K/V 投影与 FFN 的输入投影拼成较大的矩阵乘；分支之间没有前后依赖，也给并发执行留下机会。

它不是等价的重排

在串行块中，F 的输入依赖 A；并行形式中本层 FFN 还不能利用本层注意力刚获取的信息，要到后续层才看到合成结果。因此并行残差块改变模型函数，并不是把两个独立操作调换执行次序就完成的无损优化。具体是否共享 norm、使用一个还是两个 γ，必须查看所复现模型的定义。

系统收益为何需要实测

第28页的论文摘录讨论较大模型下的训练加速，这依赖融合矩阵乘和硬件调度。若两个分支都已占满 GPU，简单放到两个 stream 可能只是竞争资源；并行结构还可能增加同时存活的激活。第29页总结了调查中较常见的串行选择，表达的是采用频率，并未推翻并行结构在特定规模的收益。这里的“并行层”也不同于跨设备的张量并行或流水线并行。

本节参考（完整书目见讲义末尾）：[slides] · [palm]

Many variations in position embeddings

Sine embeddings: add sines and cosines that enable localization

$$Embed(x, i) = v_x + PE_{pos}$$

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Notable models:

Original transformer

Absolute embeddings: add a position vector to the embedding

$$Embed(x, i) = v_x + u_i$$

Notable models:

GPT1/2/3, OPT

Relative embeddings: add a vector to the *attention computation*

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

Notable models:

T5, Gopher, Chinchilla

Rope embeddings (next slides..)

Notable models:

GPTJ, PaLM, LLaMA

Most 2024+ models

第 3 讲 · 原页 30–30 · 中文讲解

位置编码的四种入口

为什么内容向量还不够

没有位置机制的自注意力，面对同一组 token 的不同排列缺少显式顺序信号；因果 mask 会限制可见前缀，但不能替代丰富的位置表示。绝对加性位置编码将词向量 v_x 与位置向量 u_i 相加。u_i 可以是可学习表，也可以是固定的正余弦：不同通道使用不同频率，使位置形成多尺度信号。可学习表通常需要处理超出已训练位置范围的情况；正余弦能计算任意 i，也不保证长于训练长度的效果。

相对位置不只有一种实现

相对位置方法让注意力分数显式依赖 i–j。有的方法给分数加按相对距离查表的标量，有的方法把相对向量加入 key 或 value。第30页画的是向量进入 attention 的示意，而 T5 使用相对距离分桶的偏置，因此不能把这一行公式当成所有列出模型的逐字实现。RoPE 则旋转已投影的 Q/K，使点积以一种特定形式依赖相对位置。

编者补充：比较时看三个问题

位置信息是加在最初嵌入、每层 attention 的分数，还是 Q/K 向量中？对所有位置一起平移时，显式位置项如何变化？超过训练长度时，是没有参数可查，还是虽然可计算但相位/距离分布已变？这三个问题比记住“绝对/相对”的标签更有用。下一组将证明 RoPE 的点积关系，同时说明这种关系没有自动解决上下文外推。

本节参考（完整书目见讲义末尾）：[slides] · [transformer] · [rope] · [t5]

RoPE: rotary position embeddings

High level thought process: a *relative* position embedding should be some $f(x, i)$ s.t.

$$\langle f(x,i), f(y,j) \rangle = g(x, y, i-j)$$

That is, the attention function *only* gets to depend on the relative position (i-j). How do existing embeddings not fulfill this goal?

- **Sine:** Has various cross-terms that are not relative

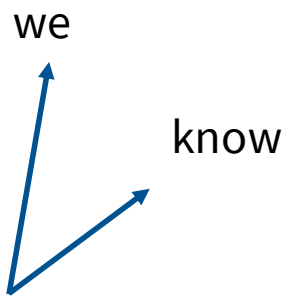
$$\langle Embed(x,i), Embed(y,i) \rangle = \langle v_x, v_y \rangle + \langle PE_i, v_y \rangle \ldots$$

- **Absolute:** obviously not relative
- **Relative embeddings:** $e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$ is not an inner product

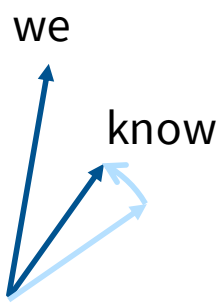
RoPE: rotary position embeddings

How can we solve this problem?

- We want our embeddings to be invariant to absolute position
- We know that inner products are invariant to arbitrary rotation.



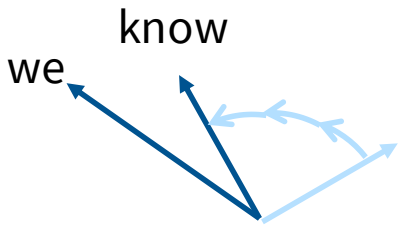
Position independent embedding



Embedding
“we know that”

Rotate we by ‘0 positions’

know by ‘1 positions’



Embedding
“of course we know”

Rotate we by ‘2 positions’

Rotate know by ‘3 positions’

第 3 讲 · 原页 31–32 · 中文讲解

RoPE 的目标：让点积显式依赖相对位移

加法为什么会产生交叉项

为了看清结构，先忽略 Q/K 投影。加性编码后的内积是 $(x+p_i)^T(y+p_j)=x^Ty+x^Tp_j+p_i^Ty+p_i^Tp_j$。即使纯位置项 $p_i^Tp_j$ 可以只与 i–j 有关，中间两项仍把内容与各自绝对位置耦合。这里不是说模型不能学出相对关系，而是相对性没有由结构直接保证。第31页的重复 i 下标应结合上下文按两个位置 i、j 理解。

编者补充：从二维旋转推导

取列向量约定，$R(\varphi)=[[\cos\varphi,-\sin\varphi],[\sin\varphi,\cos\varphi]]$，$R(\varphi)^TR(\psi)=R(\psi-\varphi)$。位置 i 的 query 是 $q_i'=R(i\theta)q$，位置 j 的 key 是 $k_j'=R(j\theta)k$，于是 $(q_i')^Tk_j'=q^TR(i\theta)^TR(j\theta)k=q^TR((j-i)\theta)k$。显式位置因素只剩相对位移 j–i。课件用 i–j 也可以，通过相反旋转约定或改写函数 g 吸收符号；实现内部一致才是关键。

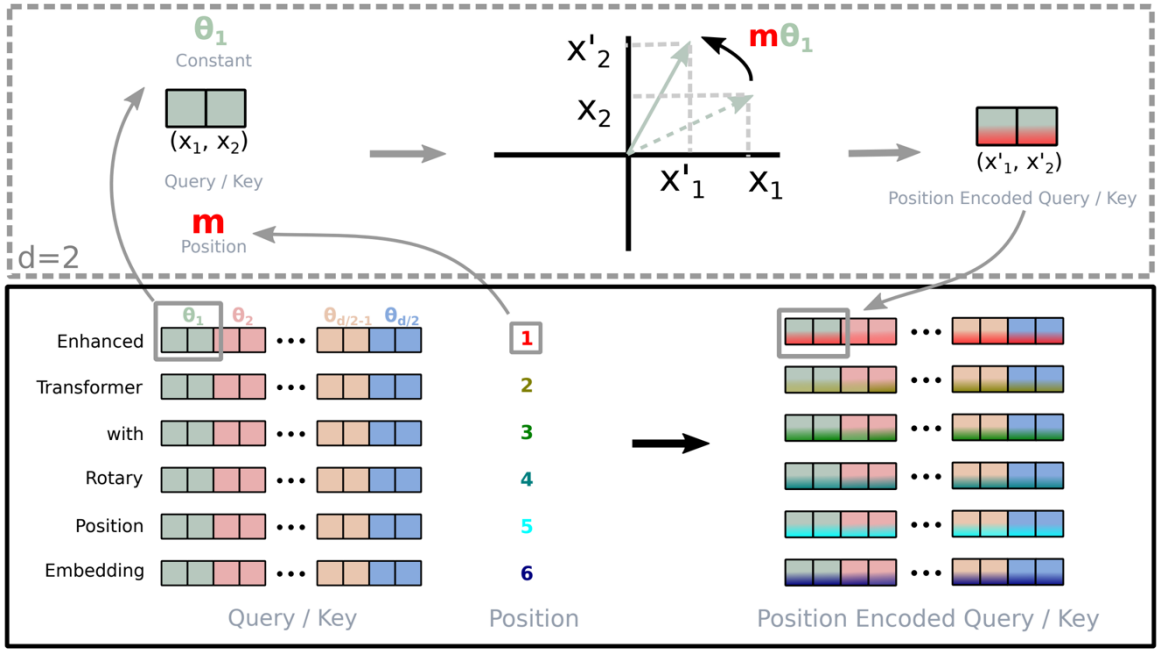
对应“we know”图的手算

令 $q=k=(1,0)$，每位置旋转 $\theta=\pi/2$。位置(0,1)的点积是 $\cos(\pi/2)=0$；将两词同时移到(2,3)，点积仍为0。若两词距离改成2，点积为$\cos\pi=-1$。图中一起旋转的两个箭头夹角不变，对应共同平移不改变显式相对关系。注意 q、k 本身来自上下文相关隐藏状态，换一段前缀可能改变内容向量；RoPE 不保证真实模型在任意文本平移后整体输出不变。

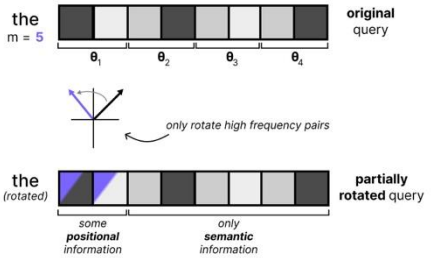
本节参考（完整书目见讲义末尾）：[slides] · [rope]

RoPE: rotary position embeddings

There are many rotations, which one do you pick?



[Su et al 2021]



Gemma 4 alternative: just first 2

Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)

The actual RoPE math

Multiply with sines and cosines

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_{\{q,k\}}\boldsymbol{x}_m \tag{14}$$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{15}$$

Difference with sine embeddings – not additive, no cross terms

第 3 讲 · 原页 33–34 · 中文讲解

多频率 RoPE 的矩阵与实现

编者补充：把二维旋转推广到一个头

设每头维度 k 为偶数，取坐标对 (x_0,x_1)、(x_2,x_3)…，第 r 对使用角频率 θ_r=base^(−2r/k)，r=0,…,k/2−1。位置 i 的旋转角为 iθ_r；将每个2×2旋转矩阵放到对角线上，就得到第34页的大矩阵。低编号对变化快，高编号对变化慢，因此一个向量同时携带多个相对距离尺度。base 常见示例为 10000，但实际配置会改它；不要把示例当所有模型固定值。

展开后无需构造稠密矩阵

一对坐标 (a,b) 的输出是 (a cosφ−b sinφ, a sinφ+b cosφ)。逐对算只需 O(k) 操作，没必要构造 k×k 矩阵并做 $O(k^2)$ 乘法。例：k=4、base=10000，则 $\theta_0=1$、$\theta_1=0.01$，位置2的两个旋转角分别为2弧度与0.02弧度，不是2度与0.02度。每个旋转块都正交，所以在精确算术下保持向量范数。

实现变体与外推边界

有的实现把前后半段配对，而不是相邻坐标配对；只要权重与坐标排列约定一致，它们可以表示同类旋转。也有只在部分通道使用 RoPE 的设计，剩余通道保留内容信息。第33页右侧展示一种局部旋转的近期例子，不应据它改变基础推导。可以为任意位置算 sin/cos，不意味着模型见过相应相位组合；长上下文涉及训练长度、频率调整和任务测试，旋转的正交性本身不是长度泛化定理。

本节参考（完整书目见讲义末尾）：[slides] · [rope]

Implementation and code for RoPE

Usual attention stuff

Get the RoPE matrix cos/sin

Multiply query/key inputs

```
query_states = self.q_proj(hidden_states)
key_states = self.k_proj(hidden_states)
value_states = self.v_proj(hidden_states)

# Flash attention requires the input to have the shape
# batch_size x seq_length x head_dim x hidden_dim
# therefore we just need to keep the original shape
query_states = query_states.view(bsz, q_len, self.num_heads, self.head_dim).transpose(1, 2)
key_states = key_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)
value_states = value_states.view(bsz, q_len, self.num_key_value_heads, self.head_dim).transpose(1, 2)

cos, sin = self.rotary_emb(value_states, position_ids)
query_states, key_states = apply_rotary_pos_emb(query_states, key_states, cos, sin)
```

…

Same stuff as the usual multi-head self attention below

Note: embedding at *each attention operation* to enforce position invariance

第 3 讲 · 原页 35–35 · 中文讲解

把 RoPE 放进注意力代码的正确位置

按原代码图追踪形状

原代码先把 hidden states 投影为 query/key/value，再 reshape 成头，取得相应 position IDs 的 cos/sin，最后对 query 和 key 应用旋转。以普通多头为例：[b,n,d] → [b,n,h,k] → [b,h,n,k]；旋转沿最后一维成对进行，位置表沿 n 广播，头和 batch 共享同一位置的角度。之后 QK^T 得 [b,h,n,n] 分数，softmax 沿 key 的序列维进行。

编者补充：缓存解码为什么容易出错

训练或 prefill 中位置通常是0…n−1；后续生成新 token 时应使用其全局递增位置，而不是每次给单 token 输入重新编号为0。一般可把已旋转的 K 和未旋转的 V 写入 cache，下次直接读；不要每步再次旋转所有旧 K，否则会重复施加角度。若框架使用另一种缓存约定，需要对应调整。GQA 下 Q 有 h 个头、K/V 有 g 个头，但每头旋转维度 k 仍要兼容。

范围提醒与最小正确性检查

经典 RoPE 只旋转 Q 和 K，V 不因这一步额外旋转。每层注意力都在各自投影后处理位置，不只是词嵌入层做一次。可以在 CPU 检查：旋转前后范数相等；两位置共同平移后，相同原始 q/k 的点积保持；整段计算与逐步缓存计算在相同 mask 和位置编号下相符。这里列出最后一项作为实现建议，本讲没有运行模型级缓存一致性测试。

课堂补充 · **00:37:57.750–00:38:53.960**

第三方录像字幕转述：RoPE 通过乘正余弦构造旋转，避免加性位置向量产生的交叉项；实际在注意力投影后对 Q/K 应用，而且每次注意力计算都这样处理。这里的相对性指固定内容向量下的位置机制，并不取消上下文内容本身的变化。

本节参考（完整书目见讲义末尾）：[slides] · [rope] · [captions]

Hyperparameters

Transformer hyperparameter questions you might have had in 224n..

- How much bigger should the feedforward size be compared to hidden size?
- How many heads, and should num_heads always divide hidden size?
- What should my vocab size be?

And other model setting questions

- Do people even regularize these huge LMs?
- How do people scale these models - very deep or very wide?

Surprising (?) consensus hyperparameter 1

Feedforward – model dimension ratio.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\boldsymbol{d_{ff} = 4\, d_{model}}$$

This is *almost always* true. There's just a few exceptions.

Exception #1 – GLU variants

Remember that GLU variants scale down by 2/3rd. This means most GLU variants have $d_{ff} = \frac{8}{3} d_{model}$. This is mostly what happens. Some notable such examples.

Model	d_{ff}/d_{model}
PaLM	4
Mistral 7B	3.5
LLaMA-2 70B	3.5
LLaMA 70B	2.68
Qwen 14B	2.67
DeepSeek 67B	2.68
Yi 34B	2.85
T5 v1.1	2.5

Models are roughly in this range, though PaLM, LLaMA2 and Mistral are slightly larger

第 3 讲 · 原页 36–38 · 中文讲解

FFN 宽度：4d 与 8d/3 从哪里来

先区分模型宽度与中间宽度

d 是贯穿各层残差流的维数，m=d_ff 是 FFN 内部展开后的维数。残差相加要求 FFN 最后返回 d，却没有要求内部一定为4d；4是长期有效的经验比例。每个 token 独立经历 d→m→d 的变换，不会因为 m 较大就能看到更多 token。扩大 m 改变的是通道表达能力、参数和逐 token 计算成本。

编者补充：把预算算到每层

普通两矩阵 FFN 参数约2dm，前向矩阵乘约4bndm FLOPs（一次乘加计2）。m=4d 时，参数为 $8d^2$。门控 FFN 有两条 d→m 的分支和一条 m→d 的回投，参数3dm，矩阵乘约6bndm FLOPs；令 m=8d/3 同样得到$8d^2$参数和约$16bnd^2$前向 FLOPs，另有逐元素激活与相乘。标准 h k=d 的多头注意力四个投影约$4d^2$参数，所以常见块忽略 norm/bias 后约$12d^2$，其中 FFN 占约2/3。这不包括词嵌入，也不包括 attention 的 n^2计算。

如何读第38页的例外表

表中许多门控模型靠近2.67倍，也有3.5或4倍。若 d=4096，8d/3≈10922.67 不为整数，模型必然需要取整，往往还为 kernel 选择某个倍数；因此实际比例稍偏离不是公式算错。较大的门控 m 可能是主动分配更多参数，而不是忘了2/3规则。面对一份配置，先确定它的 FFN 有两块还是三块矩阵，再比较比例，不能只拿 d_ff 的数字横向排序。

本节参考（完整书目见讲义末尾）：[slides] · [glu]

Exception #2 – T5

As we have (and will) see, most LMs are have boring, conservative hyperparameters. One exception is T5 [Raffel et al 2020] which has some *very bold* settings.

In particular, for the 11B model, they set

$$d_{ff} = 65{,}536$$
$$d_{model} = 1024$$

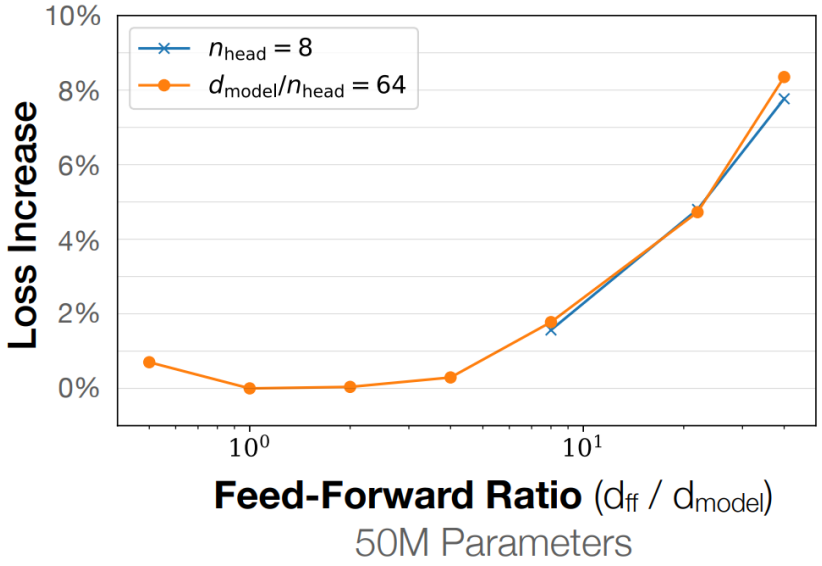
For an astounding 64-times multiplier.

> for "11B" we use $d_{\text{ff}} = 65{,}536$ with 128-headed attention producing a model with about 11 billion parameters. We chose to scale up d_{ff} specifically because modern accelerators (such as the TPUs we train our models on) are most efficient for large dense matrix multiplications like those in the Transformer's feed-forward networks.

Other, recent exceptions – Gemma 2 (8x), SmolLM/Gemma 3/Gemma 4 (4x, GLU)

Why this range of multipliers?

Empirically, there's a basin between 1-10 where this hyperparameter is near-optimal



[Kaplan+ 2020]

What can we learn from the model-dim hyperparam?

- The ‘default’ choices of $d_{ff} = 4d_{model}$ and $d_{ff} = 2.66d_{model}$ have worked well for nearly all modern LLMs.

- But T5 does show that even radical choices of $d_{ff} = 64d_{model}$ can work. This hyperparameter choice isn’t written in stone.

- That said, T5 has a follow-up model (T5 v1.1) that is ‘improved’ and uses a much more standard 2.5 multiplier on GeGLU, so the 64-times multiplier is likely suboptimal.

第 3 讲 · 原页 39–41 · 中文讲解

FFN 比例的经验盆地与 T5 例外

极端例子说明形状并不被默认值锁死

第39页的 T5-11B 用 d=1024、m=65536，比例64；这仍满足矩阵连接要求。单个普通 FFN 的矩阵参数是2×1024×65536=134,217,728，而同 d 下 m=4096 只有8,388,608，相差16倍。较窄残差流与极宽 FFN 是一种不同的预算分配，也反映原报告对硬件大矩阵效率的考虑；并不意味着64倍在任意任务都更好。

第40页的曲线在比较什么

横轴是 m/d，纵轴是损失增加百分比，图注对应约50M参数实验；中间一段较平坦，两端升高。横轴采用宽范围展示，不能把几乎同高的像素当成统计上完全等价。比较比例通常意味着其他维度也要调整以约束预算；极窄 FFN 可能限制逐 token 处理，极宽 FFN 则挤占其他组件。该图支持先在合理范围内选默认值，而不是花昂贵搜索寻找一个虚假的精确常数。

编者补充：不要把跨版本进步当单因素实验

第41页指出 T5 v1.1 使用较常见的 GeGLU 比例，暗示早期64倍不是唯一合理路线。但版本间还改变了激活和训练设定，所以“后续版本更好”不能单独证明原比例导致了全部损失。实用策略是从4d普通 FFN 或约8d/3门控 FFN 起步，在系统或预算目标明确时再偏离，并报告改变后参数总数和训练成本。

本节参考（完整书目见讲义末尾）：[slides] · [t5] · [scaling]

Surprising (?) consensus hyperparameter 2

Head-dim*num-heads to model-dim ratio. As a reminder, slide from 224n.

Multi-head self-attention is computationally efficient

- Even though we compute h many attention heads, it's not really more costly.
  - We compute $XQ \in \mathbb{R}^{n \times d}$, and then reshape to $\mathbb{R}^{n \times h \times d/h}$. (Likewise for XK, XV.)
  - Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
  - Almost everything else is identical, and the **matrices are the same sizes.**

This doesn't *have to* be true: we can have head-dimensions > model-dim / num-heads.

But most models do follow this guideline

How many heads, whats the model dim?

Some examples of this hyperparameter

	Num heads	Head dim	Model dim	Ratio
GPT3	96	128	12288	1
T5	128	128	1024	16
T5 v1.1	64	64	4096	1
LaMDA	128	128	8192	2
PaLM	48	258	18432	1.48
LLaMA2	64	128	8192	1
Qwen 3.5 (27B)	24	256	5120	1.2

Most models have ratios around 1 – notable exceptions by some google models.

第 3 讲 · 原页 42–43 · 中文讲解

头数乘头维度是否必须等于模型宽度

编者补充：形状约束的真实形式

把所有查询头的总宽度记为 H=h k。只要 W_Q∈R^(d×H)、W_K/W_V 使用相容的头维度、输出投影 W_O∈R^(H×d)，残差块就能返回 d。H=d 是常见节省预算的设定，不是注意力公式的公理。若 H=d 且各头等宽，才要求 h 整除 d；允许 H≠d 时，并不要求 h 整除 d。以 d=768、h=8、k=128 为例，H=1024，拼接头输出后乘1024×768的 W_O 仍合法。

选择更多头同时改变哪些成本

在标准多头、Q/K/V 宽度都为 H 时，四个投影参数共4dH。固定 H 而增加 h，会减小 k，矩阵投影参数不变；但显式保存的 attention 分数仍有 bhn^2 个元素，头数多会增加这部分内存。改变 h 还改变每个头的表达子空间，不能把它看成完全无意义的 reshape。后文 GQA 让 K/V 总宽度小于 Q，因此又需要分别记录 h 与 g。

原稿表格的算术核对

第43页 GPT3 的96×128=12288，比例1；T5 的128×128/1024=16，说明 H 可以远大于 d。PaLM 行写成48头、每头258、d=18432、比例1.48，这些数不自洽：按前3列算约0.672，并非1.48。这里保留原表并提示不要将该行作为可复现配置。Qwen 行24×256/5120=1.2则在算术上自洽。复现实际模型时应查对应规模的原报告或配置，别用这张调查表替代实现规范。

本节参考（完整书目见讲义末尾）：[slides]

Aspect ratios

Should my model be deep or wide? *How* deep and how wide?

Most models are surprisingly consistent on this one too!

Sweet spot?

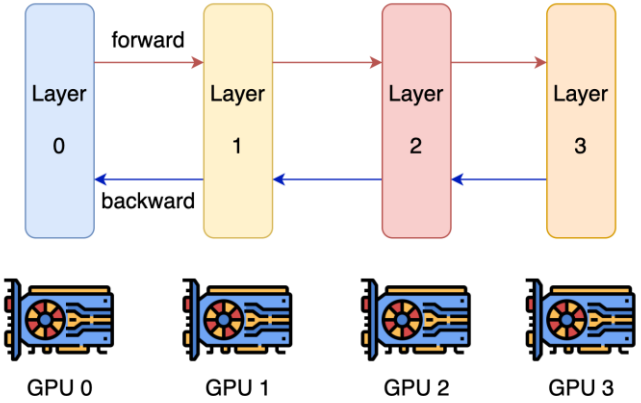
Model	d_{model}/n_{layer}
BLOOM	205
T5 v1.1	171
PaLM (540B)	156
GPT3/OPT/Mistral/Qwen /OLMo 3	128
LLaMA / LLaMA2	102
Gemma 3	87
Gemma 4	61
T5 (11B)	33

Considerations about aspect ratio

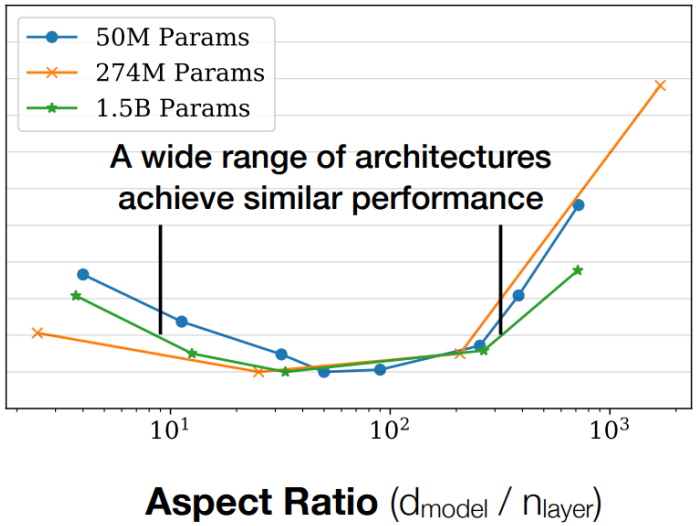
Extremely deep models are harder to parallelize and have higher latency

> **The Limits of Depth vs Width** We note an obvious limitation with our advice. Scaling depth has an obvious limiter, i.e., they are non-parallelizable across different machines or devices and every computation has to always wait for the previous layer. This is unlike width, which can be easily parallelizable over thousands or hundreds of thousands of devices. Within the limitation of scaling

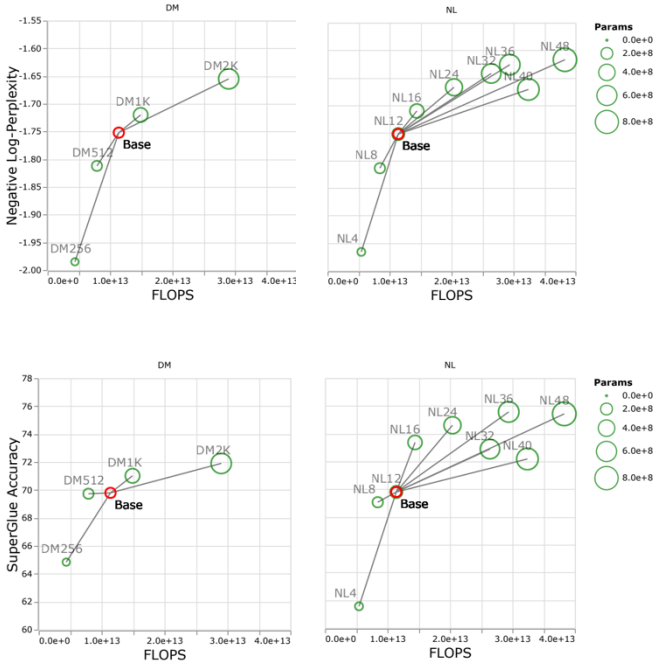
[Tay et al 2021]



Evidence on aspect ratio scaling



[Kaplan et al 2020]



[Tay et al 2021]

第 3 讲 · 原页 44–46 · 中文讲解

深度与宽度：参数相近，延迟未必相近

比例的定义与图表的意义

这里的 aspect ratio 是 d/L，L 为层数；数值大意味着相对宽、浅。第44页列出从几十到两百左右的样本，并不是物理常数。第46页左图横轴 d/L、纵轴相对损失，显示多个规模中较宽的可行区间；右侧图涉及不同扩展方式的计算/效果比较，说明规模变化不能只看参数数。第45页的流水线图则强调一层依赖前一层，深度形成关键路径。

编者补充：同参数预算下的两种形状

忽略嵌入等项，使用上一组的常见块估算 $P \approx 12Ld^2$。L=32、d=4096 时约64.42亿参数；L=128、d=2048 也约64.42亿。后者层数四倍、每层矩阵较小，每个 token 要经过更多串行阶段。即使理论矩阵 FLOPs 接近，kernel 启动、设备通信、流水线气泡和矩阵利用率仍可能不同，单请求解码延迟也会不同。两种模型的质量也不能仅由参数相同推出相同。

为什么“越宽越好”同样不成立

更宽会增加单层激活和矩阵大小，可能需要更大的张量并行通信；更深提供更多次非线性与信息交换，未必可由一个宽层完全替代。系统上训练吞吐与推理延迟还可能偏好不同形状。课件给出的范围应作为搜索起点：先确认模型能放入目标显存，估计关键路径和通信，再在预算内比较验证损失。不要用一个 d/L 数字忽略 FFN 比例、头维度和嵌入参数。

本节参考（完整书目见讲义末尾）：[slides] · [scaling]

What are typical vocabulary sizes?

Monolingual models – 30-50k vocab

Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000

Multilingual / production systems 100-250k

Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Gemma 4	262144
DeepSeek	100000
Qwen 15B	152064
Yi	64000

Monolingual vocabs don’t need to be huge, but multilingual ones do

第 3 讲 · 原页 47–47 · 中文讲解

词表大小连接 tokenization 与模型成本

词表大不等于表示能力免费增加

V 是可输出的 token 种类数，d 是每个 token 的向量维数，二者不能混淆。嵌入矩阵有 Vd 个参数；若输入与输出权重共享，主要矩阵只计一次，否则约两份。更大的词表可能把同一段文字编码得更短，从而减少 attention 的序列成本，但输出头每个位置需在更大的 V 上产生 logits，训练数据也要足够覆盖稀有 token。

编者补充：把数量级算出来

若 d=4096，V=32000，单份嵌入参数为131,072,000；V=128000 则为524,288,000。BF16 每参数2字节时，单份权重约250 MiB 与1000 MiB，未计梯度、优化器状态和输出激活。假设新 tokenizer 将同一文本从1000 token 缩到800 token，普通全注意力的配对数大约乘0.64，而词表输出成本还受新 V 影响；不能只看 token 变少就断言总成本一定下降。

如何理解单语与多语的表

课件中单语样本常在数万，多语或产品系统常用更大词表，这是语言覆盖、压缩效率与参数成本的折中。字节回退使小词表也可表示多种语言，所以“多语必须大词表”应理解为效率动机，而非可表示性的绝对限制。不同 tokenizer 下每 token 的损失或 perplexity 不能直接横比，因为一个 token 对应的文本单位不同；比较时需统一 tokenizer 或采用每字节等共同单位。

本节参考（完整书目见讲义末尾）：[slides]

Dropout and other regularization

Do we need regularization during pretraining?

Arguments against:

- There is *a lot* of data (trillions of tokens), more than parameters.
- SGD only does a single pass on a corpus (hard to memorize)

This is all quite reasonable.. but what do people do in practice?

Dropout and weight decay in practice

Model	Dropout*	Weight decay
Original transformer	0.1	0
GPT2	0.1	0.1
T5	0.1	0
GPT3	0.1	0.1
T5 v1.1	0	0
PaLM	0	(variable)
OPT	0.1	0.1
LLaMA	0	0.1
Qwen 14B	0.1	0.1

Many older models used dropout during pretraining

Newer models (except Qwen) rely only on weight decay

* Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout. This may not be true of closed models.

第 3 讲 · 原页 48–49 · 中文讲解

预训练还需要正则化吗

先分清 **dropout** 与 **weight decay**

Dropout 在训练时随机屏蔽激活，常按保留率重标以保持期望；推理时关闭。Weight decay 在参数更新中收缩权重，既可影响函数复杂度，也会改变优化轨迹。课件的直觉是：海量 token、接近单遍训练时，传统小数据反复拟合的过拟合情形不一定占主导。但 token 数多于参数数不是严格泛化保证，文本内部相关、重复片段和稀有样本仍可能被记忆。

第**49**页表格怎样读

旧模型不少采用0.1 dropout，新一些模型常设为0，同时仍用 weight decay；也有例外。表脚注指出很多论文没有详细写 dropout，公开实现能辅助判断，但闭源模型的“没写”并不等于已证实“没有”。数据表是样本记录，不是规定每个后续模型都该照做。微调数据少、多轮训练、领域分布窄时，dropout 的取舍可能与大规模预训练不同。

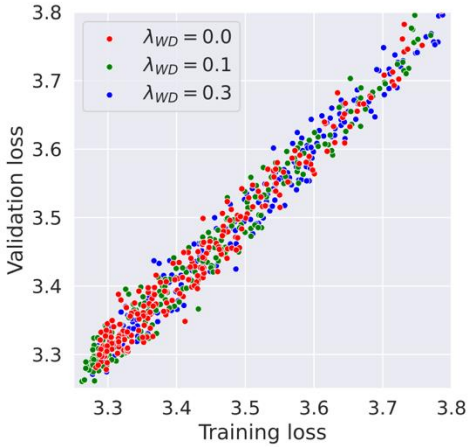
编者补充：避免把两种现象混在一起

如果 weight decay 让训练损失和验证损失一起下降，这更接近优化改善；如果训练拟合稍差但验证更好，才更像常见的防过拟合叙事。两者可以并存，应画 train/validation 曲线并控制 token 预算，而不是仅凭“正则化”这个名字推测作用。下一页讨论的是特定单遍语言建模实验中的优化解释。

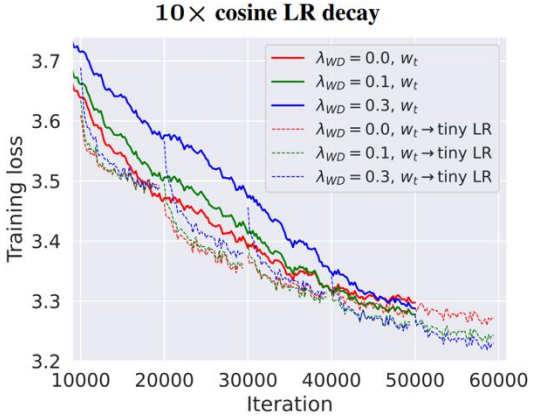
本节参考（完整书目见讲义末尾）：[slides]

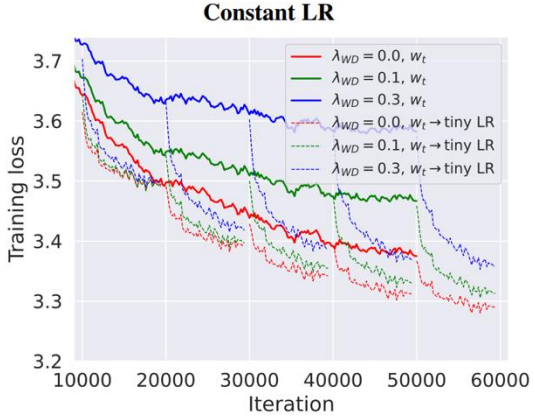
Why weight decay LLMs?

[Andriushchenko et al 2023] has interesting observations about LLM weight decay



It's not to control overfitting





Weight decay interacts with learning rates (cosine schedule)

Summary: hyperparameters

Feedforward

- Factor-of-4 rule of thumb (8/3 for GLUs) is standard (with some evidence)

Head dim

- Head dim*Num head = D model is standard – but low to no validation

Aspect ratio

- Wide range of ‘good’ values (100-200). Systems concerns dictate the value

Regularization

- You still ‘regularize’ LMs but its effects are primarily on optimization dynamics

Name	Year	MLP factor	Aspect ratio (d/layer)	weight decay	drop_rate
Original transformer	2017	4	85	0	0.1
GPT	2018	4	64	0.1	0.1
T5 (11B)	2019	64	43	0	0.1
GPT2	2019	4	33	0.1	0.1
T5 (XXL 11B) v1.1	2020	2.5	171	0	0
mT5	2020	2.5	171	0	0
GPT3 (175B)	2020	4	128	0.1	0.1
GPTJ	2021		146	0.1	0
LaMDA	2021	8	128		
Anthropic LM (not claude)	2021	4	128		
Gopher (280B)	2021	4	205		
GPT-NeoX	2022	4	140	0.01	0
BLOOM (175B)	2022	4	205	0.1	0
OPT (175B)	2022	4	128	0.1	0.1
PaLM (540B)	2022	4	156		0
Chinchilla	2022	4	102		
Baichuan 2	2023	2.68	128	0.1	0
Mistral (7B)	2023	3.5	128	0.1	0
LLaMA2 (70B)	2023	3.5	102	0.1	0
LLaMA (65B)	2023	2.6875	102	0.1	0
GPT4	2023		0		
Olmo 2	2024	2.6875	128		
Gemma 2 (27B)	2024	8	100		
Nemotron-4 (340B)	2024	4	192		0
Qwen 2 (72b) - same for 2.5	2024	3.609	102		
Falcon 2 11B	2024	4	68	0.1	
Phi3 (small) - same for phi4	2024	3.5	128		
Llama 3 (70B)	2024	3.5	102		0
Reka Flash	2024		0		
Command R+	2024	2.75	192		
OLMo	2024	2.6875	128	0.1	0
Qwen (14B)	2024	2.675	128	0.1	0.1
DeepSeek (67B)	2024	2.6875	86	0.1	0
Yi (34B)	2024	2.857142	119	0.1	0
Mixtral of Experts	2024		0		
Command A	2025		0		
Gemma 3	2025	4	87		
SmolLM2 (1.7B)	2025	4	85		

第 3 讲 · 原页 50–51 · 中文讲解

Weight decay 与学习率计划相互作用

图中支持的结论

第50页左图把训练损失与验证损失配对，不同 weight decay 的点接近一条线，说明该实验中的验证改善伴随着训练目标改善。中图使用衰减学习率，右图为恒定学习率，不同衰减强度的相对表现发生变化。重点是 weight decay 不能脱离学习率计划独立讨论，而不是宣称它在所有任务中都绝不具有正则化效果。原研究的语言模型实验属于具体规模、语料与训练时长。

编者补充：从更新式看相互作用

简化的 AdamW 可写 $\theta_{t+1}=(1-\eta_t\lambda)\theta_t-\eta_t u_t$，$u_t$ 是经过动量和二阶矩预条件的梯度方向，λ 为衰减系数。若忽略梯度，经过多步后的权重是 $\theta_0\times\Pi_t(1-\eta_t\lambda)$，当 $\eta_t\lambda$ 很小时近似 $\theta_0 \exp(-\lambda\Sigma_t\eta_t)$。因此相同 λ 在不同学习率计划下，累计收缩也不同。例如 $\eta=0.001$、$\lambda=0.1$ 时单步因子0.9999，1000步仅收缩项约为0.9048；这不是完整训练预测，只隔离一个机制。

把这部分还原成可执行判断

AdamW 的解耦衰减也不能简单等同于把 $\lambda||\theta||^2/2$ 加进 Adam 的损失，因为后者的 $\lambda\theta$ 会进入梯度预条件。实际还常对 bias 或 norm 参数使用不同参数组。本讲未指定完整训练配方；第51页总结的 FFN、头宽和 aspect ratio 默认值，需要与学习率、初始化、数据和系统一同验证。参数形状合理只是训练的起点。

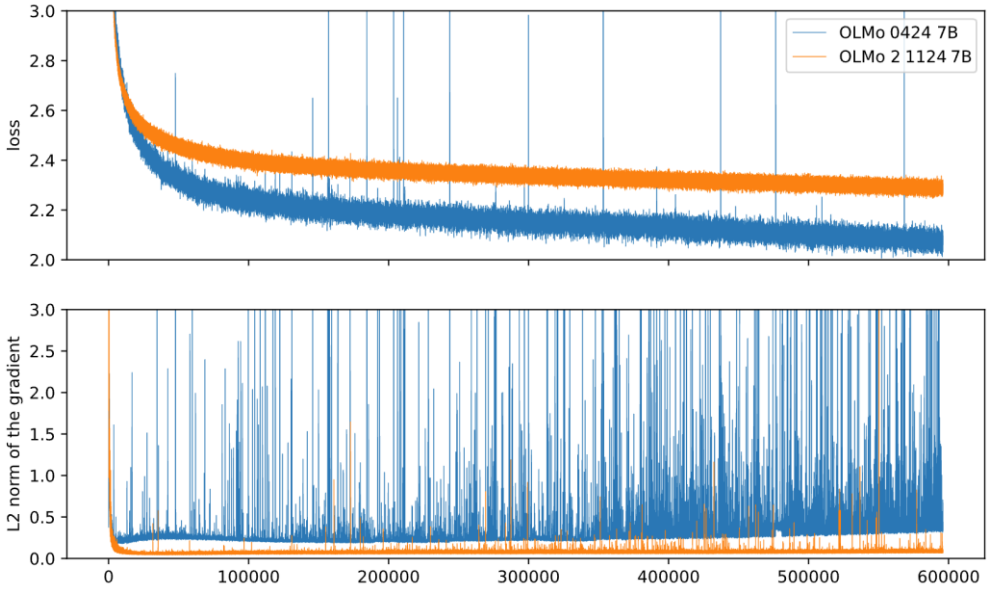
课堂补充 · **01:01:36.230–01:03:13.830**

第三方录像字幕转述：讲者用单遍语言模型实验说明 weight decay 可能主要改善优化，并强调它与学习率衰减结合时的行为不同于恒定学习率。这个课堂观察有实验设置边界，不应扩大成所有任务中 weight decay 都不会防止过拟合。

本节参考（完整书目见讲义末尾）：[slides] · [weight_decay] · [captions]

Stability tricks

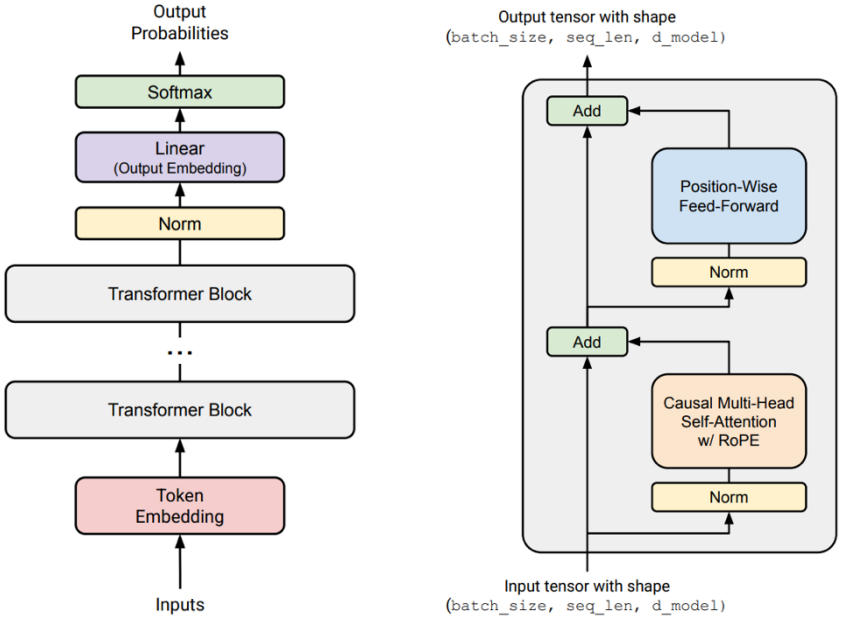
Recently, lots of attention on *stable training*



Don’t train models that look like the blue curve!

Where do the issues arise? Beware of softmaxes!

Softmaxes – can be ill-behaved due to exponentials / divison by zero



第 3 讲 · 原页 52–53 · 中文讲解

训练尖峰与 softmax：数值问题和优化问题

从曲线定位问题层次

第52页上图是损失随训练推进的变化，下图展示梯度范数的频繁尖峰。曲线平滑与最终损失低不是同一回事；一个模型可以平均损失较低却频繁出现异常步。第53页指出两个 softmax 位置：每头注意力在可见 token 上归一化，输出头在词表上归一化。它们的候选集合、输入尺度和作用不同，后续稳定化方法不能不加区分地互换。

编者补充：先修正数值实现

softmax(z)_i=exp(z_i)/Σ_jexp(z_j)。若 z=(1000,999)，直接求指数可能溢出；先减最大值 m=1000，得到 exp(0) 与 exp(−1)，概率约(0.7311,0.2689)。由于分子分母共同乘 exp(−m)，精确概率不变。logsumexp(z)=m+logΣ_jexp(z_j−m)，交叉熵可直接用 logsumexp(z)−z_y 计算，避免先求极小概率再取 log。

减最大值不是全部解决方案

若一行 attention 被完全 mask 成负无穷，直接做“负无穷减负无穷”仍产生 NaN，需保证至少有合法位置或对这种行作专门处理。即使 softmax 数值有限，极大的 logit 差仍会使分布极尖、梯度饱和；这是尺度与优化问题，不是单靠减最大值能解决的。训练发散也可能来自数据异常、学习率或精度，不能见到尖峰就认定 softmax 是唯一原因。

本节参考（完整书目见讲义末尾）：[slides]

Output softmax stability – the ‘z-loss’

Recall the softmax calculation

$$\begin{aligned} \log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \Sigma_{r'=1}^{|V|} e^{U_{r'}(x)} \end{aligned}$$

$$\begin{aligned} L &= \sum_i \left[\log(P(x_i)) - \alpha(\log(Z(x_i)) - 0)^2\right] \\ &= \sum_i \left[\log(P(x_i)) - \alpha \log^2(Z(x_i))\right] \end{aligned}$$

[From Devlin 2014]

This is useful for stability! PaLM used this ‘z loss’ trick.

> We additionally use an auxiliary loss of $z_loss = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2 (2025), OLMo 3 (2025)

第 3 讲 · 原页 54–54 · 中文讲解

输出 **z-loss**：约束 **logit** 的共同平移方向

先理解 **softmax** 留下的自由度

设词表 logits 为 $z_1 \ldots z_V$，$Z=\Sigma_j \exp(z_j)$，正确 token y 的负对数似然是 $\log Z - z_y$。所有 logits 加同一个 c 时，log Z 增加 c，z_y 也增加 c，两者抵消，概率与交叉熵不变。因此仅靠交叉熵，不会惩罚沿“全体 logits 一起上移”方向的漂移；但有限精度实现和其他计算可能受到较大数值尺度影响。

编者补充：目标函数与导数

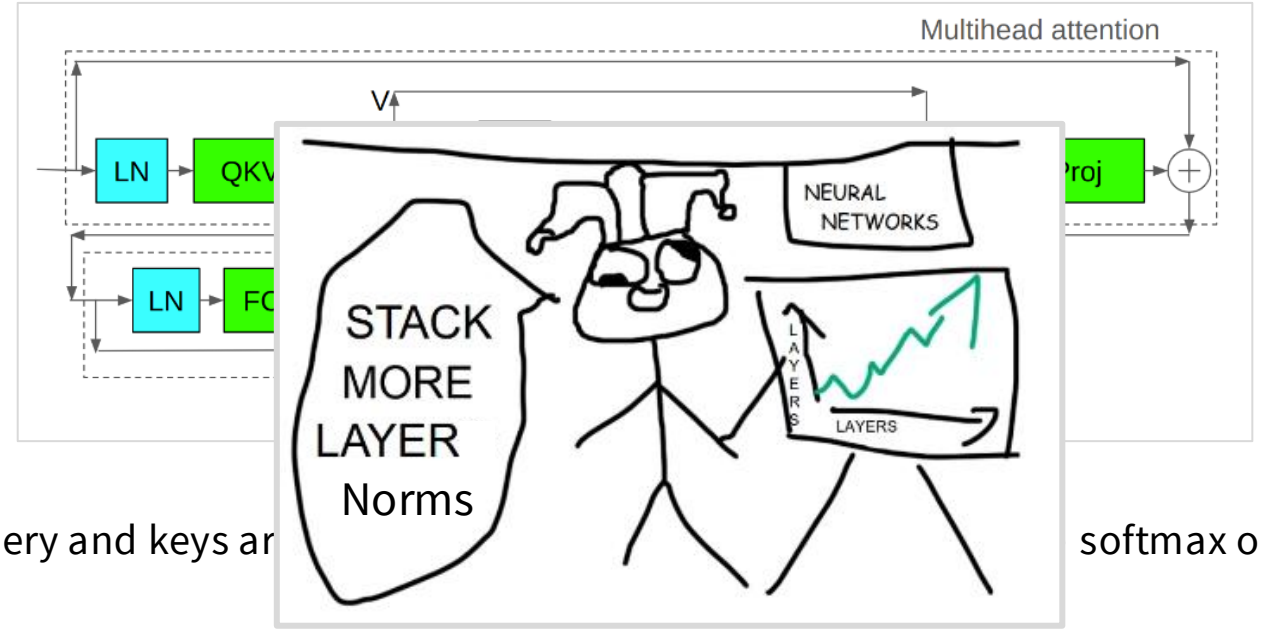
一种常见形式是 $L_{total} = L_{CE} + \alpha(\log Z)^2$，其中 $\alpha>0$ 较小。因为 $\partial \log Z/\partial z_i=p_i$，辅助项对 z_i 的梯度为$2\alpha(\log Z)p_i$：log Z 为正时整体倾向向下推，为负时向上推，目标是把 log Z 拉近0。它不是把所有 z_i 拉到0，更不是强迫均匀概率。原页上方以最大化对数似然写惩罚为减号；本文采用最小化负对数似然，所以是加号。

一个保持概率不变的例子

$z=(0,0)$ 时概率各为0.5，但 $\log Z=\log 2$。把二者减去 log2，变成$(-\log 2,-\log 2)$，概率仍是0.5，而 $\log Z=0$、辅助项为0。z-loss 偏好这种“选定共同偏移”的表示。实现时必须在原 logits 上用稳定 logsumexp 得到真正的 log Z；如果只算减最大值后的 logsumexp 而忘了加回最大值，得到的就不是同一惩罚。它改变训练目标，不能与不改变概率的数值 max-shift 混为一谈。

本节参考（完整书目见讲义末尾）：[slides] · [palm]

Attention softmax stability – the ‘QK norm’



The query and keys ar softmax operation.

Other examples: DCLM, OLMo2, Gemma 2, Qwen3, OLMo 3, Gemma 4

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]

Logit soft-capping.

Soft-capping the logits to some maximum value via Tanh

> **Logit soft-capping**. We cap logits (Bello et al., 2016) in each attention layer and the final layer such that the value of the logits stays between −soft_cap and +soft_cap. More specifically, we cap the logits with the following function:
>
> $$\text{logits} \leftarrow \text{soft_cap} * \tanh(\text{logits}/\text{soft_cap}).$$
>
> We set the soft_cap parameter to 50.0 for the self-attention layers and to 30.0 for the final layer.

Prevents logits from blowing up, but also might have perf issues?

Table 4: Models perplexity with confidence interval ±0.1 at 95% level.

bf16 baseline	*soft_cap*	*QKV_norm*	*QK_norm_cap*	*QK_norm*	*QK_FC_norm*
11.19	11.24	10.85	11.00	10.84	10.87

第 3 讲 · 原页 55–56 · 中文讲解

QK-Norm 与 logit soft-capping

QK-Norm 在哪一条路径上工作

注意力 logit $s_{ij}=q_i^{T}k_j/\sqrt{k}$。Q/K 的范数增长会放大分数差，使注意力分布变尖。QK-Norm 在进入点积前，对投影后的 query/key 再作 LayerNorm 或 RMSNorm；它不同于对整个残差流作 Pre-Norm，因为 W_Q、W_K 仍可改变投影后的尺度。实际 norm 覆盖哪些轴、是否有可学习 gain、在 RoPE 前后如何安排，应以模型定义为准。

编者补充：范数控制能保证什么

暂忽略 ε 和可学习 γ，逐头 RMSNorm 后 $||q||_2\approx\sqrt{k}$、$||k||_2\approx\sqrt{k}$，Cauchy–Schwarz 给出 $|q^{T}k/\sqrt{k}|\le\sqrt{k}$。这解释了去除任意输入幅值增长的作用，但并未保证分数永远很小，也没有保证某个固定温度；学习到的 gain 还会改变范围。它与余弦注意力相关，却不完全等于把所有向量归一化到单位长度再直接点积。

soft-capping 的公式与梯度

第56页的另一思路直接把分数 z 变成 $c\cdot\tanh(z/c)$，$c>0$。当 $|z|\ll c$ 时 $\tanh(z/c)\approx z/c$，几乎不变；当 $|z|$ 很大时输出趋近 $\pm c$。导数为$1-\tanh^2(z/c)$，因此极端值区域梯度也会变小。例：$c=2$、$z=10$，输出约 1.9998、导数约0.00018，数值受控的同时也可能抑制纠正极端 logit 的速度。输出 logits 与 attention logits 可用不同 cap，并非必须同时使用。

方法之间的边界

QK-Norm 作用在向量尺度，soft-capping 直接改变分数，z-loss 在训练损失中惩罚输出归一化常数，max-shift 则只是数值等价变换。它们处理的是不同层次。注意 soft-cap 应与 mask 正确组合：若先把 $-\infty$ mask 也压成有限的 $-c$，未来位置可能重新获得概率；通常先处理真实 logits，再施加因果 mask。课件的性能提示还涉及 kernel 支持，某项更稳定也可能更慢，需要实测。

本节参考（完整书目见讲义末尾）：[slides] · [gemma2]

Attention heads

Most models don’t touch the attention heads much at all with a few minor exceptions..

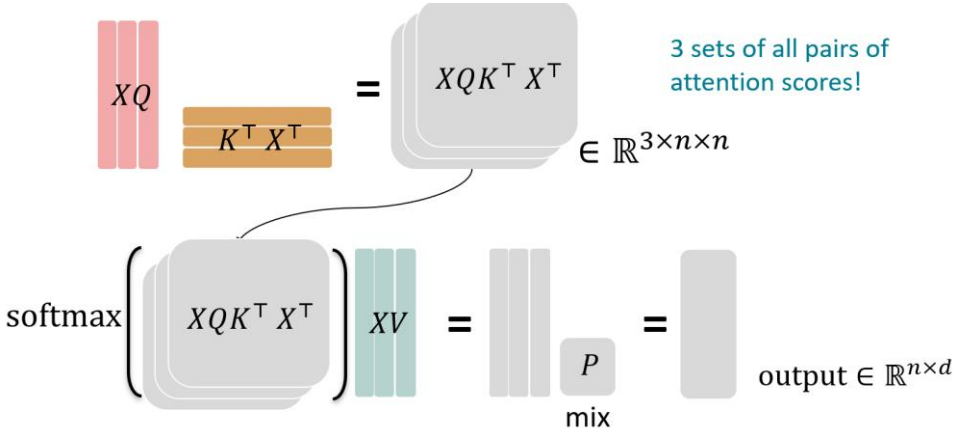
GQA / MQA : Saving inference costs by reducing the number of heads

Sparse or sliding window attention (GPT4/Mistral): restricting the attention pattern to reduce compute cost

Exotic SSM stuff (Jamba, Falcon 3, Qwen 3.5, etc): next lecture!

GQA/MQA – Reducing attention head cost

Let's think about the compute involved for attention



d = hidden dim
b = batch
n = length (<d)
h = heads
k = head dim (d/h)

X softmax projection

Total arithmetic operations (bnd^2), **total memory accesses** $(bnd + bhn^2 + d^2)$

Arithmetic intensity (compute/memory) is high $O\left(\left(\frac{1}{k} + \frac{1}{bn}\right)^{-1}\right)$ - we can keep our GPUs running

第 3 讲 · 原页 57–58 · 中文讲解

注意力成本：先分析整段训练或 prefill

为什么先区分三个维度

从这里开始使用原页记号：b 为 batch，n 为序列长度，d 为模型宽度，h 为头数，k=d/h 为每头宽度。训练时已知整段 token，可并行计算所有 query；prefill 也能一次处理输入提示词。解码时下一个 token 尚未生成，不能像 prefill 那样把未来 query 一起送入矩阵乘。下面只估单层 attention 主项，FFN、反向传播和输出词表头需要另算。

编者补充：从矩阵形状得到运算与访存

Q/K/V 与输出投影总计 $O(bnd^2)$ 运算；QK^T 和乘 V 共 $O(bn^2d)$。因此一般总项是 $O(bnd^2+bn^2d)$。若 $n<d$，原页用 $O(bnd^2)$ 作为主导简化。朴素实现会搬运 $O(bnd)$ 激活、$O(bhn^2)$ 分数与 $O(d^2)$ 权重，故按相同元素字节数计，算术强度近似为 $bnd^2/(bnd+bhn^2+d^2)=1/[1/d+n/(kd)+1/(bn)]$。这是由该访存模型直接得到的式子。

怎样理解原页更简短的式子

原页写 $O((1/k+1/(bn))^{-1})$，是结合 $n<d$ 与忽略次要项后的粗略尺度表达，不应把它当精确恒等式或任意 n 的紧界；例如 $n\ll d$ 时把 $n/(kd)$ 替成$1/k$会丢失重要差别。物理单位若为 FLOPs/byte，还要计乘加常数与每元素字节数。真实 FlashAttention 可避免把整个 $n\times n$ 分数矩阵写回 HBM，因此 bhn^2 的访存项也会改变；计算 n^2 关系本身并不会因此自动消失。此处的目的是建立读懂课件近似式的条件。

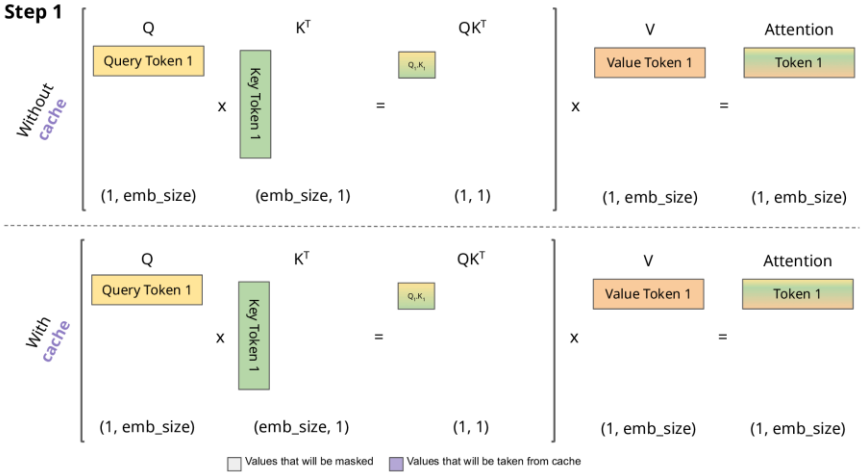
本节参考（完整书目见讲义末尾）：[slides] · [mqa]

GQA/MQA – Reducing attention head cost

What about the *incremental* case when we generate text?

Key difference: can't parallelize the generation process – needs to be step by step

In this case – we need to incrementally re-compute/update attention via the 'KV cache'



[Animation from https://medium.com/@joaolages/kv-caching-explained-276520203249]

GQA/MQA – Reducing attention head cost

What's the incremental arithmetic intensity?

Total arithmetic operations (bnd^2), **total memory accesses** $(\overset{\text{K, V}}{bn^2d} + \overset{\text{projection}}{nd^2})$

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - need large batches + short seq length (n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.

第 3 讲 · 原页 59–60 · 中文讲解

KV cache 与逐 token 解码的成本

缓存保存什么，为什么不缓存所有 query

生成位置 t 时，旧 token 的 K/V 在因果 decoder 中不会依赖未来 token，因此可复用；本步只需从新隐藏状态得到 q_t、k_t、v_t，将新 K/V 追加到缓存，再用 q_t 去读所有旧 K/V。旧 query 对计算新输出没有直接用途，所以标准 cache 保存 K/V。原页动画展示的是把整段反复重算变成“复用历史+追加一个位置”，不是消除对历史 K/V 的读取。

编者补充：先算一步，再求累计

对历史长度 t，普通 MHA 单步投影计算 $O(bd^2)$，读历史并做注意力为 $O(btd)$，访存主项约 $O(d^2+btd)$。生成到长度 n，$\Sigma_t\, t=n(n+1)/2$，所以累计计算 $O(bnd^2+bn^2d)$，累计搬运约 $O(nd^2+bn^2d)$。$n<d$ 时保留计算主项 $O(bnd^2)$，相除得到 $I\approx 1/(1/b+n/d)$。第60页的 bnd^2 与 bn^2d 是累计 n 步的记法，不能当成一步的成本；单步只须把 n 换成当前历史 t，并去掉累计因子。

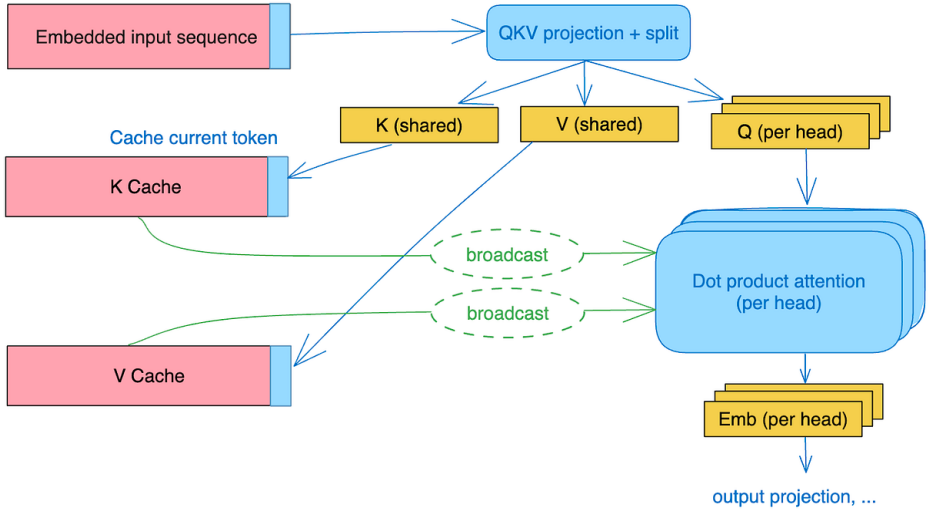
数字例子：带宽为什么成为瓶颈

忽略常数，b=1、d=4096、n=2048 时 $I\approx 1/(1+0.5)=0.667$ 个运算单位/元素搬运；b=16 时约 $1/(0.0625+0.5)=1.778$。增大 batch 可以摊薄权重读取，但 n/d 项仍在，继续增加 b 收益受限。这里不是测得的 FLOPs/byte，更不是精确性能预测，旨在显示长历史的 KV 读取不会凭 batch 消失。长上下文、缓存量化、权重驻留与硬件层级还会改变实际瓶颈。

本节参考（完整书目见讲义末尾）：[slides] · [mqa]

MQA – just have fewer key dimensions.

Key idea – have multiple queries, but just one dimension for keys and values



We have much fewer items to move in and out of memory (KV Cache)

Total memory access $(bnd + bn^2k + nd^2)$, **Arithmetic intensity** $O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$

[figure from https://blog.fireworks.ai/multi-query-attention-is-all-you-need-db072e758055]

第 3 讲 · 原页 61–61 · 中文讲解

MQA：保留多组查询，共享一组 K/V

“一组”不等于“一维标量”

普通 MHA 有 h 组 Q、h 组 K、h 组 V；MQA 仍有 h 组 query，但只有1组 key 和1组 value，每组仍是 k 维向量。不同 query 仍产生不同 attention 权重，只是从同一套历史 K/V 取信息。第61页的“one dimension”是简略说法，应理解为把 KV 头轴缩为1，不是把向量宽度 k 变成1。最终 h 个头的输出仍能拼接再投影。

编者补充：缓存和带宽如何减少

标准 MHA 每层每序列每个 token 存2hk个元素，MQA 只存2k个，所以理想 KV cache 容量与相关搬运可减少 h 倍。Q 和输出投影仍在，attention 分数仍有 h 个查询头，模型总计算或总显存不会自动少 h 倍。以 d=4096、h=32、k=128、b=1、n=4096、L=32、每元素2字节为例：MHA 缓存 $2bnLhk\times2=2{,}147{,}483{,}648$字节，即2 GiB；MQA 为64 MiB，未计元数据、对齐和其他状态。

把课件的强度式接上上一页

在同一累计生成近似中，搬运量可写 $O(bnd+bn^2k+nd^2)$，对应当前激活、共享 KV 历史、每步权重。主计算仍取 $O(bnd^2)$，相除得到 $I\approx[1/d+n/(dh)+1/b]^{-1}$。与 MHA 的 n/d 项相比，KV 项缩为 n/(dh)，所以可提高算术强度。只有实现避免把共享 K/V 真正复制成 h 份并有效复用加载，才能兑现带宽收益；逻辑广播不应直接等于物理缓存复制。

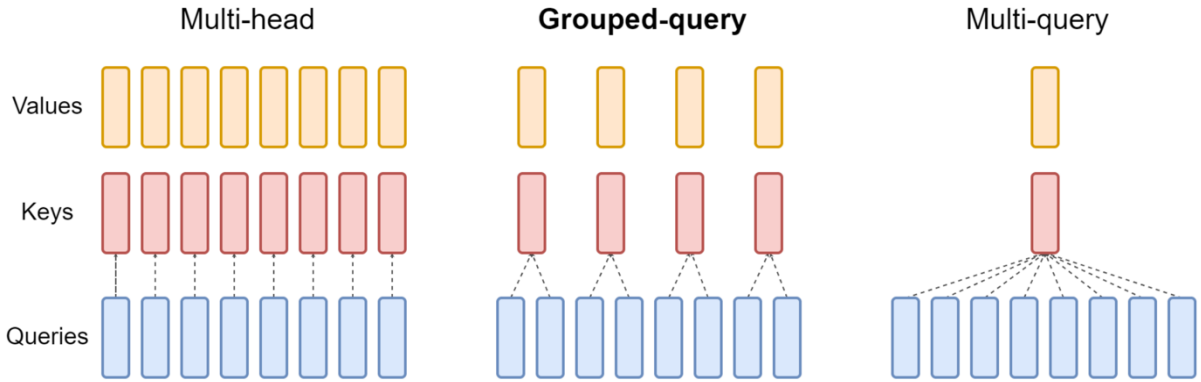
课堂补充 · **01:19:32.420–01:20:36.750**

第三方录像字幕转述：多个 query 保持独立，所有头共享 K/V，因而减少缓存搬运并提高算术强度。字幕在约01:20:13先出现“降低算术强度”的说法，随后约01:20:29立即纠正为“提高”；本节按纠正后的结论与公式解释，不把前一句孤立当作证据。

本节参考（完整书目见讲义末尾）：[slides] · [mqa] · [captions]

Additional extensions – GQA

Don’t go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

More recently – MLA (multihead latent attention) from deepseek v2

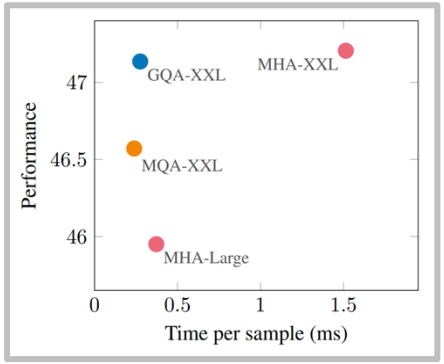
Does MQA hurt? Sometimes..

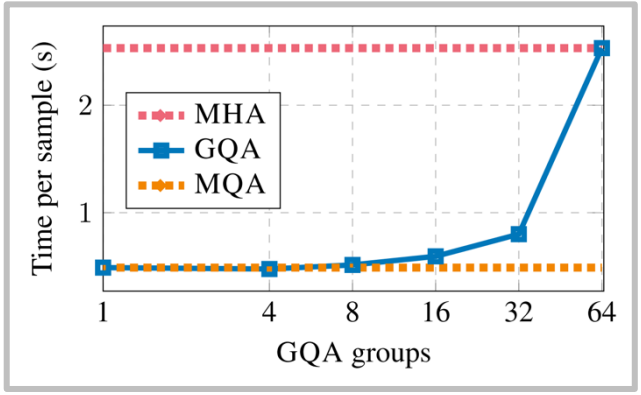
Small PPL hit w/ MQA [Shazeer 2019]

Table 3: Billion-Word LM Benchmark Results.

Attention	h	d_k, d_v	d_{ff}	dev-PPL
multi-head	8	128	8192	**29.9**
multi-query	8	128	9088	30.2
multi-head	1	128	9984	31.2
multi-head	2	64	9984	31.1
multi-head	4	32	9984	31.0
multi-head	8	16	9984	30.9

Low/no hit w/ GQA [Ainslie 2023]





第 3 讲 · 原页 62–63 · 中文讲解

GQA：在 MHA 与 MQA 之间选择 KV 头数

按原图从左向右读

第62页左图 MHA 每个 query 头配一组独立 K/V，中图 GQA 将若干 query 头分组共享 K/V，右图 MQA 让所有 query 共用一组。设查询头数 h、KV 头数 g，常见等分实现要求 h 能被 g 整除；g=h 回到 MHA，g=1 回到 MQA。这里的“减头数”主要减 KV 头，而非把所有 query 头都一起删掉。

编者补充：**GQA** 的内存公式

每层缓存形状可看作 K、V 各 [b,g,n,k]，总字节为2bnLgk s，s 为每元素字节数。延续上一页 h=32 的例子，g=8 时缓存为512 MiB，较2 GiB 的 MHA 少4倍，较64 MiB 的 MQA 大8倍。查询头仍32 个，QK 分数的头轴不随 g 缩为8。K/V 投影参数各为 dgk，Q 与输出投影各约 dhk；这一拆分是检查模型配置与显存估算时的关键。

第**63**页的质量与速度权衡

左表显示原 MQA 实验可能有小的 perplexity 损失，右图把质量分数与每样本时间结合，并给出 GQA 组数变化的耗时。perplexity 越低越好，任务分数则常越高越好，先确认坐标含义再找折中点。图支持某些 GQA 配置接近 MHA 质量、同时接近 MQA 的速度优势，不意味着所有 g 或所有任务都无损。原 GQA 工作还讨论从多头 checkpoint 转换后继续训练；仅把旧 K/V 权重平均并直接部署，不等同于已完成这一适配过程。

与 **MLA** 的关系边界

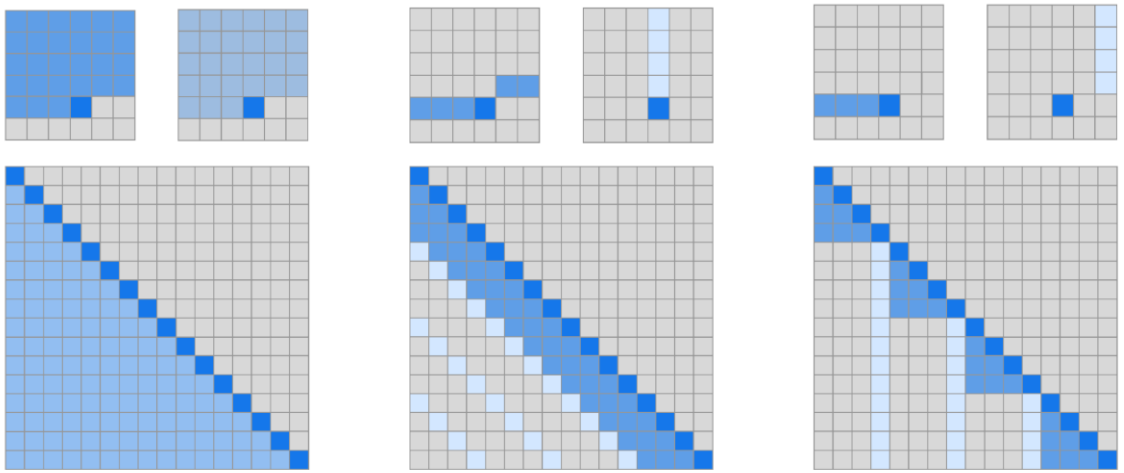
原页最后提到 MLA，它通过潜在表示等机制压缩缓存，属于另一条路线；不能把它机械归类为“把 g 设得更小”。本讲只用它提示推理内存还有更多设计空间，完整计算图、位置分量与重构成本留待专门材料讨论。

本节参考（完整书目见讲义末尾）：[slides] · [gqa] · [mqa]

Sparse / sliding window attention

Attending to the entire context can be expensive (quadratic).

Build sparse / structured attention that trades off expressiveness vs runtime (GPT3, GPT-OSS, Gemma4)



(a) Transformer (b) Sparse Transformer (strided) (c) Sparse Transformer (fixed)

[Child et al 2019]

第 3 讲 · 原页 64–64 · 中文讲解

稀疏与滑动窗口注意力：减少可见连接

读懂蓝色方格图

图的行表示 query 位置，列表示 key 位置，蓝格表示允许读取的连接。普通因果注意力保留下三角；局部、步长或分块模式只保留部分连接，从结构上减少每个 query 的候选集合。它改变可表达的信息路径，不是仅把同样的密集计算换一种 kernel。FlashAttention 通常仍计算完整指定注意力模式，和减少连接数是不同轴的优化。

编者补充：窗口成本与感受野

若每个位置最多看包括自身在内的 w 个最近 token，总注意力配对约 nw 而非 n^2，忽略头数后的算术量是 O(bnwd)。单层无法直接读取超过 w−1 距离的信息，但叠 L 层后，理论上信息最多沿局部边传播约 L(w−1) 个位置。例如 w=3，每层最多向前跨2个位置，两层可形成跨4位置的路径；这不保证模型能完好保存或检索那么远的内容。

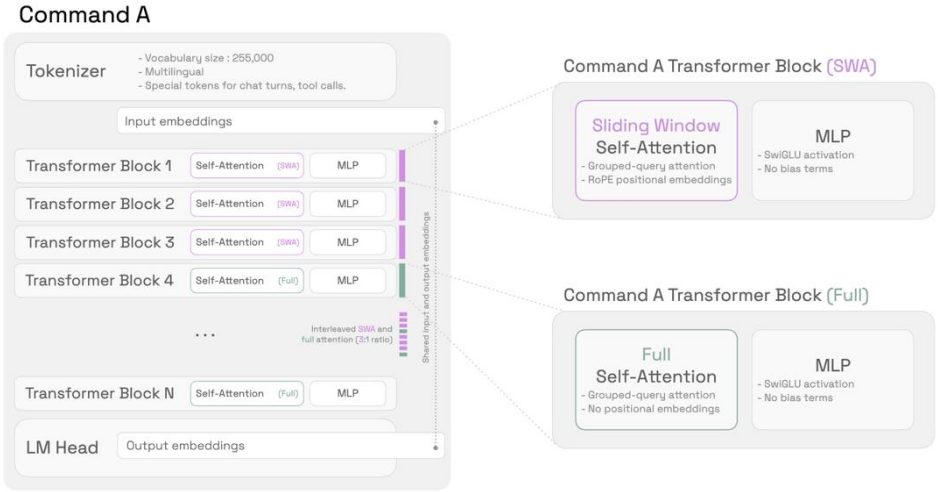
稀疏形状与硬件需要一起设计

任意稀疏矩阵虽然零多，若索引不规则，内存访问与 kernel 调度可能让运行反而更慢。块状或规则滑窗更容易利用硬件。纯滑窗解码可只保留活跃窗口的 KV，但模型若还包含 full-attention 层，那些层仍需要长历史。远距离事实检索、多段代码依赖和全局归纳都可能受局部限制，应按目标任务检查质量，而非只报告理论复杂度降低。

本节参考（完整书目见讲义末尾）：[slides] · [sparse]

Current standard trick – interleave ‘full’ and ‘LR’ attention

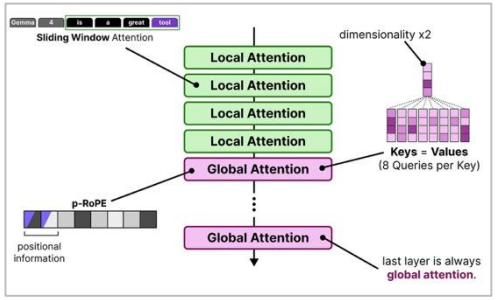
From Cohere Command A – Every 4th layer is a full attention



Long-range info via NoPE, short-range info via RoPE + SWA.

Other models – LLaMA 4, Gemma 3, Gemma 4, OLMo 3 does SWA+Full RoPE.

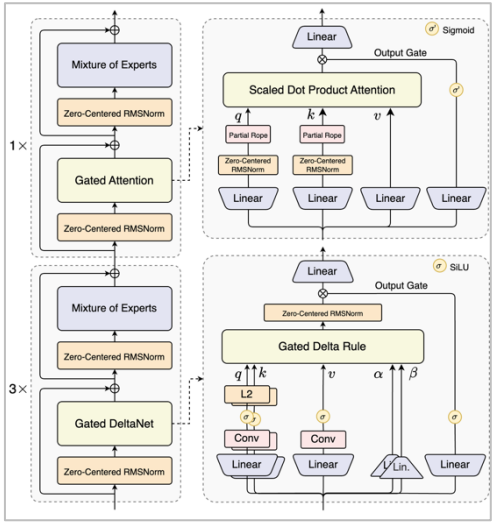
Other recent examples of interleaved attention



Gemma 4

Gradient clipping	1.0
Z-loss weight	10^{-5}
Weight decay on embeddings	No
Sliding window attention	3/4 of layers; 4,096 tokens
RoPE scaling	YaRN on full attn. layers
RoPE θ	$5 \cdot 10^5$
Layer norm applied to	Outputs

Olmo 3



Qwen 3.5 / Qwen 3 Next

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-gemma-4

第 3 讲 · 原页 65–66 · 中文讲解

交错全局与局部层：让远程信息定期流通

第65页的具体机制

Command A 的图示每4层中有3层滑动窗口注意力、1层全注意力。局部层使用 RoPE，全局层使用 NoPE，即不显式施加位置编码；这是该报告的具体组合，不能推成所有混合模型的要求。NoPE 也不等于模型完全不知道顺序：因果可见关系、其他层以及输入表示仍携带结构。全局层让相隔很远的 token 有直接连接，局部层则降低多数层的计算与缓存负担。

编者补充：先估量级，再看实现

若全局层占1/4，局部层占3/4，常见数量级估算写作$(1/4)n^2+(3/4)nw$；但要报告实际“合法配对比例”，应保留因果三角常数。对于包括自身在内的窗口，单个全局层有 $n(n+1)/2$ 个合法配对；当 $n\geq w$，局部层有 $nw-w(w-1)/2$ 个。取 n=8192、w=1024，全局为33,558,528对，局部为7,864,832对，混合后相对全局约为$0.25+0.75\times7,864,832/33,558,528=42.577\%$。只按 n^2 与 nw 粗估会得到34.375%，两者口径不同。这里仍只比较配对数量，不是整模型时间：投影、FFN、kernel 和通信都未计入。解码缓存也需分层算，全局层存完整历史，局部层能否释放旧缓存取决于实现。

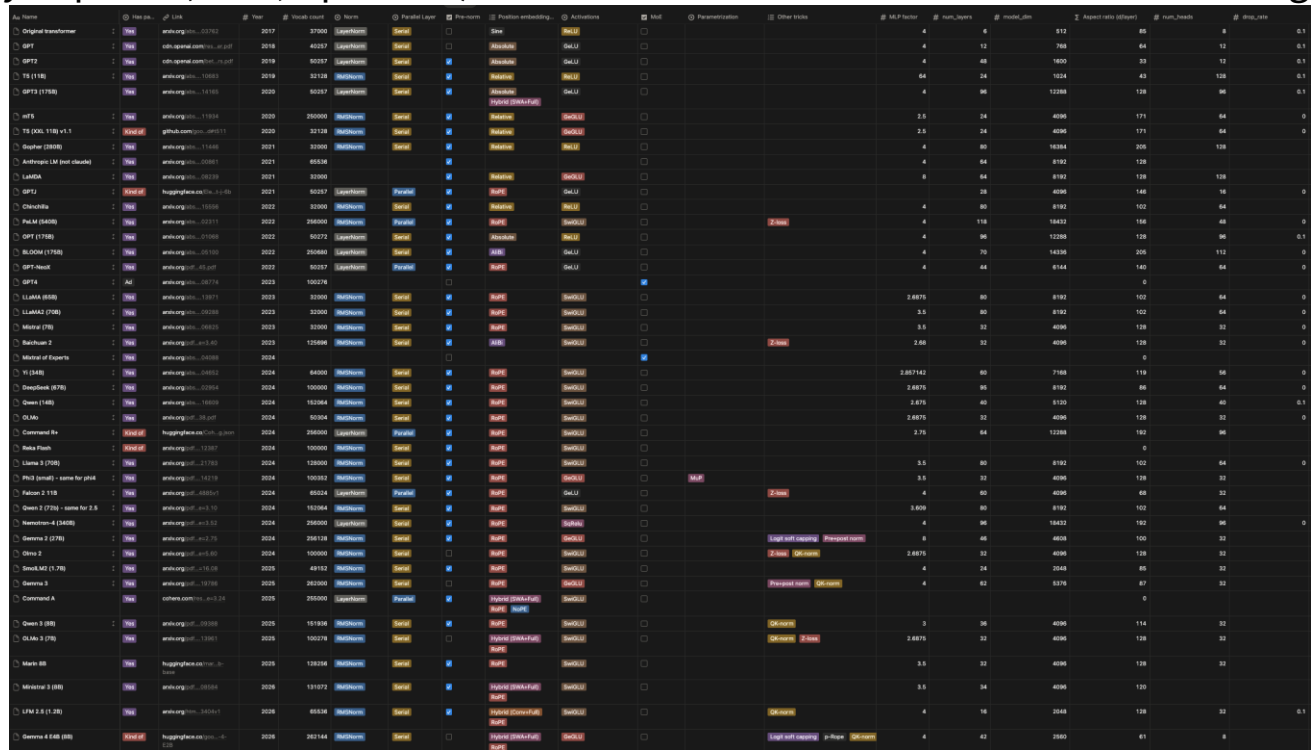
第66页的三个例子应怎样比较

Gemma、OLMo 的图表展示不同的局部/全局层配比与窗口，Qwen 例子还混合了不同类型的序列处理模块。这一页原图引用第三方可视化，因此这里只按图解释“按层组合不同信息路径”的思想，不把图中文字当成未经核对的新配置规范。混合全局与滑窗属于稀疏可见连接的选择；将注意力与线性状态模块混合则进一步改变计算机制，下一讲会讨论。不要看到“hybrid”就假定所有模型都有同一复杂度和 cache 格式。

本节参考（完整书目见讲义末尾）：[slides] · [command_a]

Recap, conclusion, etc.

Many aspects (arch, hparams) of transformers are in common across the big LMs



Major differences? Position embeddings, activations, tokenization

第 3 讲 · 原页 67–67 · 中文讲解

把架构选择连成一次完整判断

从一个最小配置出发

读完本讲应能把模型配置还原成计算图：先写残差块的 norm 与加法次序，再确定 FFN 的矩阵数与中间宽度；为 Q、K、V 与输出投影列形状；明确 RoPE 放在哪些层和通道；根据训练或推理场景估算激活与缓存。最后才讨论某个配置是否在你的数据、硬件和预算上值得尝试。

哪些知识可以直接迁移

形状相容、参数计数、旋转内积恒等式和缓存容量公式是可检查的结构事实。哪种激活质量更好、哪个深宽比最快、是否需要 dropout、某个稳定技巧是否改善训练，则需要带条件的经验验证。本讲的共同设计提供起点，例外提醒你把默认值与约束分开。下面的自测要求独立推导并解释结果，而非只背模型名称。

本节参考（完整书目见讲义末尾）：[slides]

自测与参考答案

1. Pre-Norm 是否证明了任意深度网络都不会梯度爆炸？

没有。对单个子层 y=x+F(N(x))，Jacobian 是 I+J_FJ_N，保留恒等路径是其结构优势，但其余分支仍可能放大或抵消梯度。它也不保证无需 warmup；初始化、优化器统计、学习率和精度仍会影响训练。

2. 对 x=(1,3)，忽略 ε、令 γ=1、β=0，LayerNorm 和 RMSNorm 分别输出什么？

均值2，方差1，所以 LayerNorm 为(−1,1)。均方为5，所以 RMSNorm 为(1/√5,3/√5)≈(0.4472,1.3416)。前者输出均值0，后者通常不是。若误用二范数√10作分母，得到的是另一种尺度，少了标准 RMSNorm 的√d因子。

3. 普通 FFN 的 d=768、m=3072，要改为同矩阵参数量的 SwiGLU，m 应取多少？

普通参数2dm=4,718,592；门控参数3dm_g，令二者相等得 m_g=2048。这里忽略 bias、norm 与少量逐元素开销。若仍设m_g=3072，参数会增加50%，不能把质量增益全部归因于门控。

4. 为什么 q′=R(iθ)q、k′=R(jθ)k 的点积与共同位置平移无关？

因为 $R(i\theta)^{T}R(j\theta)=R((j-i)\theta)$，故点积为 $q^{T}R((j-i)\theta)k$。同时加上位置偏移 c 后，(j+c)−(i+c)=j−i。结论假定比较的是相同内容向量；更换真实文本前缀可能改变 q、k。

5. d=768、h=8、k=128 的注意力是否合法？残差输出怎样回到768维？

合法。所有头拼接宽度H=1024，Q/K/V 投影可从768映到1024，输出 W_O 形状1024×768，将结果映回残差宽度。只有额外要求H=d时才需要每头k=d/h。

6. 同样约64.42亿块参数，32层×4096宽和128层×2048宽为什么延迟可能不同？

$P\approx 12Ld^{2}$ 对二者相同，但后者有四倍串行层数、更多启动和通信阶段，每层矩阵也更小。模型形状影响硬件利用率与关键路径；相同参数或FLOPs都不能直接推出相同延迟。

7. 为什么不同 tokenizer 的 perplexity 不能直接横向比较？

perplexity 是每token平均负对数似然的指数；不同tokenizer把文本切成不同单位，平均分母和预测任

务都变了。可统一tokenizer或使用每字节等相同文本单位，再比较编码效率和损失。

8. max-shift 与 z-loss 是否做了同一件事？

不是。减去最大值只是softmax的数值等价计算；z-loss在目标中加$\alpha(\log\Sigma\exp z)^2$，会选择logits的共同偏移并改变训练梯度。用max-shift计算原log Z时必须加回最大值。

9. 如果把 attention mask 的负无穷也先送进 c·tanh(z/c)，会怎样？

负无穷被映成有限的−c，被屏蔽的位置可能恢复非零概率。应让最终mask仍把不允许位置排除，例如先对真实分数soft-cap，再加因果mask；同时注意整行无合法位置时的特殊处理。

10. b=1、n=4096、L=32、h=32、k=128、BF16，MHA/GQA(g=8)/MQA 缓存各多大？

公式是2bnLgks，s=2字节。MHA取g=32得到2 GiB；GQA取g=8得到512 MiB；MQA取g=1得到64 MiB。这里只计K/V本体，query头仍为32；不能据缓存缩减倍率断言整个模型同倍率变快。

11. 课件的累计解码 $O(bn^2d)$ 搬运项从哪里来？

第t步要读约O(btd)的历史K/V；从t=1到n累加，$\Sigma t=n(n+1)/2$，得到$O(bn^2d)$。这与单步O(btd)不同。MQA将这项的d=hk替成k，但权重、查询和输出投影成本仍在。

12. 每4层有1层全注意力，其他3层窗口w=1024，n=8192，能否说模型整体快三倍？

不能。精确因果计数是全局$n(n+1)/2=33{,}558{,}528$对，局部$nw-w(w-1)/2=7{,}864{,}832$对，混合后配对比约42.577%。若忽略因果三角常数，只用n^2与nw的粗成本口径，则估为34.375%，不能将它称为精确配对率。两者都不是整体实测速度，因为还有投影、FFN、通信与kernel开销。

13. 设计一次SwiGLU与普通FFN的比较，至少要固定什么，记录什么？

先声明预算口径，例如同参数与同token数，按3dm_g=2dm_plain调整中间宽度。固定语料、tokenizer、模型其余结构、优化器与学习率计划；记录训练和验证损失、异常步、step/s与总用时，条件允许时重复种子。若改成同墙钟预算，要重新解释不同步数下的质量，不能混用两种结论。

参考资料

[slides] CS336 Spring 2026 第3讲原课件（教学主源，保留67页）

https://github.com/stanford-cs336/lectures/blob/main/lecture_03.pdf

[course] Stanford CS336 Spring 2026 官方课程与讲次安排

https://cs336.stanford.edu/

[captions] 第3讲公开录像的第三方字幕缓存（非教师审校稿，仅作课堂定位）

https://jingzechen.github.io/assets/courses/stanford-cs336-s26/lectures/003/captions.source.srt

[transformer] Vaswani et al., Attention Is All You Need（原论文，架构与缩放注意力）

https://arxiv.org/abs/1706.03762

[prenorm] Xiong et al., On Layer Normalization in the Transformer Architecture（原论文，初始化梯度与预热）

https://arxiv.org/abs/2002.04745

[rmsnorm] Zhang and Sennrich, Root Mean Square Layer Normalization（原论文，**RMS**定义）

https://arxiv.org/abs/1910.07467

[glu] Shazeer, GLU Variants Improve Transformer（原论文，门控与参数匹配）

https://arxiv.org/abs/2002.05202

[rope] Su et al., RoFormer: Enhanced Transformer with Rotary Position Embedding（原论文，旋转位置编码）

https://arxiv.org/abs/2104.09864

[t5] Raffel et al., Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer（原论文，**T5**）

https://arxiv.org/abs/1910.10683

[scaling] Kaplan et al., Scaling Laws for Neural Language Models（原论文，形状扫描的实验背景）

https://arxiv.org/abs/2001.08361

[palm] Chowdhery et al., PaLM: Scaling Language Modeling with Pathways（原技术报告，并行块与**z-loss**）

https://arxiv.org/abs/2204.02311

[weight_decay] D'Angelo et al., Why Do We Need Weight Decay in Modern Deep Learning?（原论文，**2023**预印本/**2024**修订）

https://arxiv.org/abs/2310.04415

[gemma2] Gemma Team, Gemma 2: Improving Open Language Models at a Practical

Size（原技术报告，分支归一化与**soft-capping**）

https://arxiv.org/abs/2408.00118

[mqa] Shazeer, Fast Transformer Decoding: One Write-Head is All You Need（原论文，**MQA**与解码成本）

https://arxiv.org/abs/1911.02150

[gqa] Ainslie et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints（原论文）

https://arxiv.org/abs/2305.13245

[sparse] Child et al., Generating Long Sequences with Sparse Transformers（原论文，稀疏注意力）

https://arxiv.org/abs/1904.10509

[command_a] Cohere, Command A Technical Report（官方报告，第**3**页架构说明）

https://cohere.com/research/papers/command-a-technical-report.pdf

[data_movement] Ivanov et al., Data Movement Is All You Need: A Case Study on Optimizing Transformers（原论文，**FLOPs**与运行时间剖析）

https://arxiv.org/abs/2007.00072

[narang] Narang et al., Do Transformer Modifications Transfer Across Implementations and Applications?（原论文，**EMNLP 2021**消融实验）

https://aclanthology.org/2021.emnlp-main.465/